


欢迎选修《数据结构》

- 课程编号：30240184
- 授课教师：邓俊辉、尹 霞、陈键飞
- 开课单位：计算机系

《数据结构与算法》DSA学什么？

线性结构

- 01.Introduction
  用尺子丈量
- 02.Vector
  向量的排序
- 03.List
  列表的排序
- 04.Stack+Quene
  各类应用

半线性结构

- 05.Binary_Tree
  树的遍历
- 06.Binary_Search_Tree
- 07.BST_Application
- 08.Advanced_BST
  各种树及中序遍历
- 09.Dictionary
  小容器放下大空间

非线性结构

- 10.Graph
- 11.Graph_Application
  图的遍历
- 12.PQ
  只关注极值的“树”
- 13.String
  最常见的串匹配
- 14.Sorting
  永恒的排序算法

【02-03】线性结构：朴素的向量和列表

2. 向量

02-A.抽象数据结构

02-B.无序向量

02-C.有序向量

02-D.起泡排序

02-E.归并排序

02-F. 位图

3. 列表

03-A.循位置访问

03-B.无序列表

03-C.有序列表

03-D.选择排序

03-E.轮换

03-F.插入排序

03-G.归并排序

03-H. 逆序对

03-XA. 游标

无序、有序、排序

2．向 量

02-A.抽象数据结构

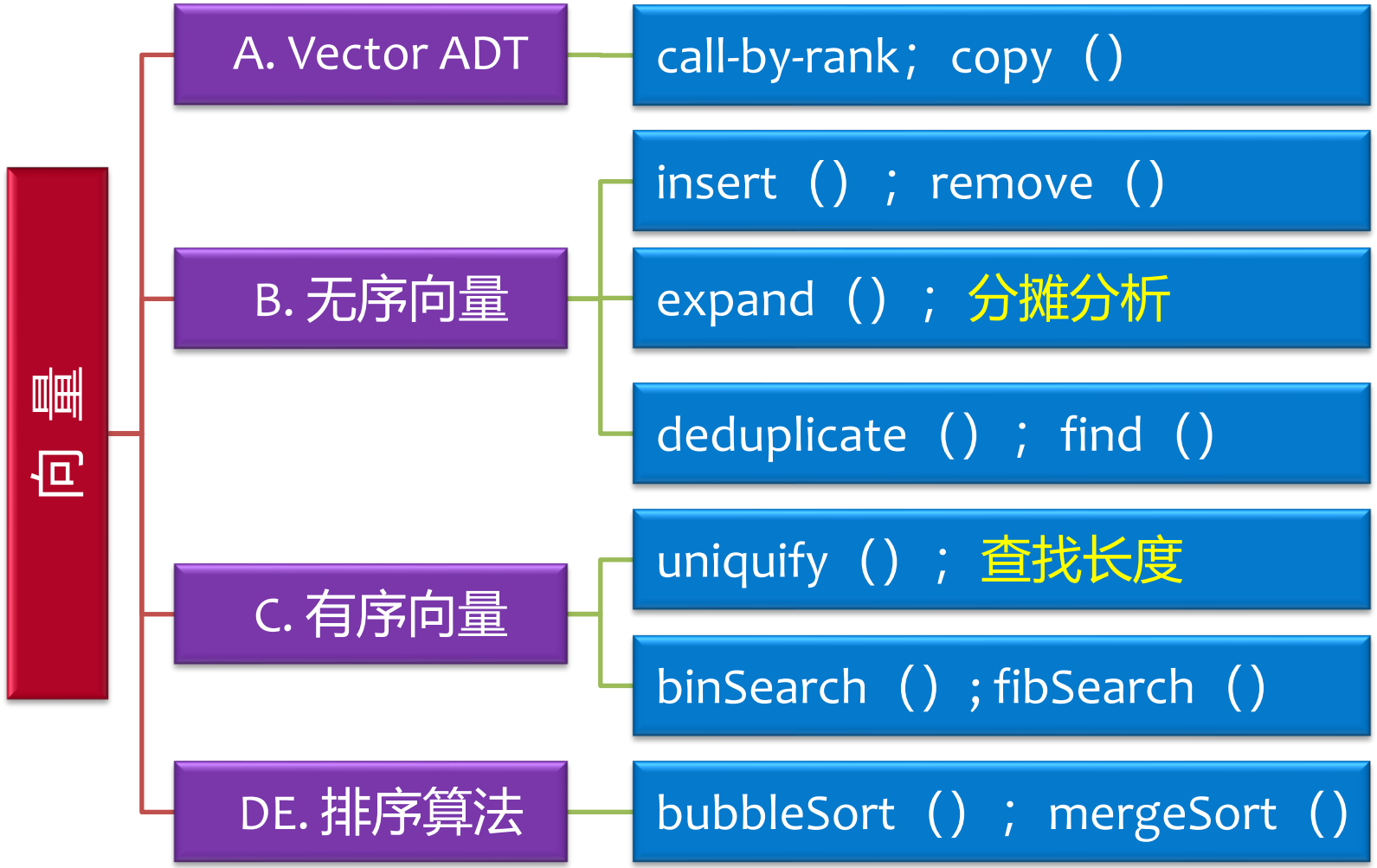
02-B.无序向量

02-C.有序向量

02-D.起泡排序

02-E.归并排序

02-F. 位图



02.Vector

A-抽象数据类型

向量

ADT：接口与实现

邓俊辉
deng@tsinghua.edu.cn

Abstract Data Type vs. Data Structure

抽象数据类型 = 数据模型 + 定义在该模型上的一组操作

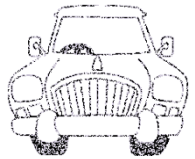
抽象定义	外部的逻辑特性	操作&语义
一种定义	不考虑时间复杂度	不涉及数据的存储方式

数据结构 = 基于某种特定语言，实现ADT的一整套算法

具体实现	内部的表示与实现	完整的算法
多种实现	与复杂度密切相关	要考虑数据的具体存储机制



Application



ADT/Interface



Implementation

Application = Interface x Implementation

- 在数据结构的具体实现与实际应用之间

  ADT就分工与接口制定了统一的规范

  - 实现：高效兑现数据结构的ADT接口操作

    //做冰箱、造汽车

  - 应用：便捷地通过操作接口使用数据结构

    //用冰箱、开汽车

- 按照ADT规范

  - 高层算法设计者可与

    底层数据结构实现者高效地分工协作

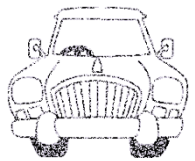
  - 不同的算法与数据结构可以便捷组合借用

  - 每种操作接口只需统一地实现一次

    代码篇幅缩短，安全性加强，软件复用度提高



Application



ADT/Interface



Implementation

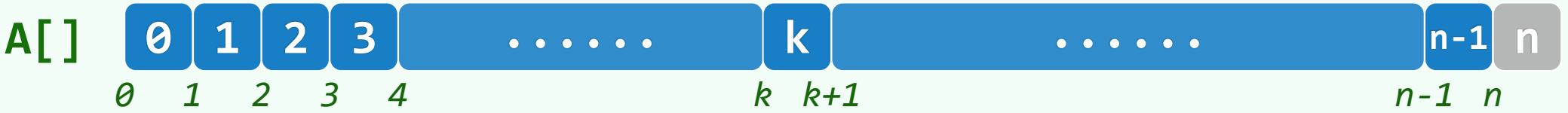
向量

ADT：从数组到向量

邓俊辉
deng@tsinghua.edu.cn

数组 ~ 循秩访问 ~ 向量

❖ C/C++语言中，数组元素与编号一一对应：A[0]，A[1]，A[2]，.....，A[n-1]



若每个元素占用的空间量为s，则A[i]的**物理地址** = A + i×s　//可能有padding

❖ 向量是数组的**抽象与泛化**，由一组元素按线性次序**封装**而成

各元素与[0,n)内的**秩**（rank）一一对应：　using Rank = int32_t; //call-by-rank

❖ 操作、管理维护更加简化、统一与安全；元素类型可灵活**选取**，便于**定制**复杂数据结构：

```
Vector<BTNodePosi<T>> child;  //B-树中，节点用向量记录所有的孩子
Vector<Vector<Edge<Te>*>> E;  //图结构中，用二维向量实现邻接矩阵，存放图中的边
```

应用者视角：Vector ADT

操作接口	功能
size() | empty()	报告元素总数/判断向量是否为空
insert(r,e)	将e作为秩为r的元素插入（后继元素顺次后移）
remove(r)	删除秩为r的元素（后继元素顺次前移）
sort() | unsort()	排序/置乱
find(e)	在无序向量中，查找与e相等者（如有多个，取最靠前者）
search(e)	在有序向量中，查找不大于e的最大者
dedup()	从无序向量中，剔除相等的元素（保留最靠后者）
uniquify()	从有序向量中，剔除相等的元素（保留最靠前者）
traverse(visit())	对每个元素统一执行visit操作

应用者视角：ADT操作实例

操作	输出	向量内容
初始化构造		空
size()	0	^
empty()	true	^
insert(0,'N')		N
insert('A')		N A
insert(0,'D')		D N A
size()	3	^
insert(1,'A')		D A N A
remove(2)	'N'	D A A
insert(2,'T')		D A T A
size()	4	^

操作	输出	向量内容
find('N')	-1	D A T A
find('T')	2	^
find('A')	3	^
dedup()	1	D A T
insert(3,'A')		D A T A
sort()		A A D T
search('A')	1	^
search('D')	2	^
search('S')	2	^
search('X')	3	^
uniquify()	1	A D T

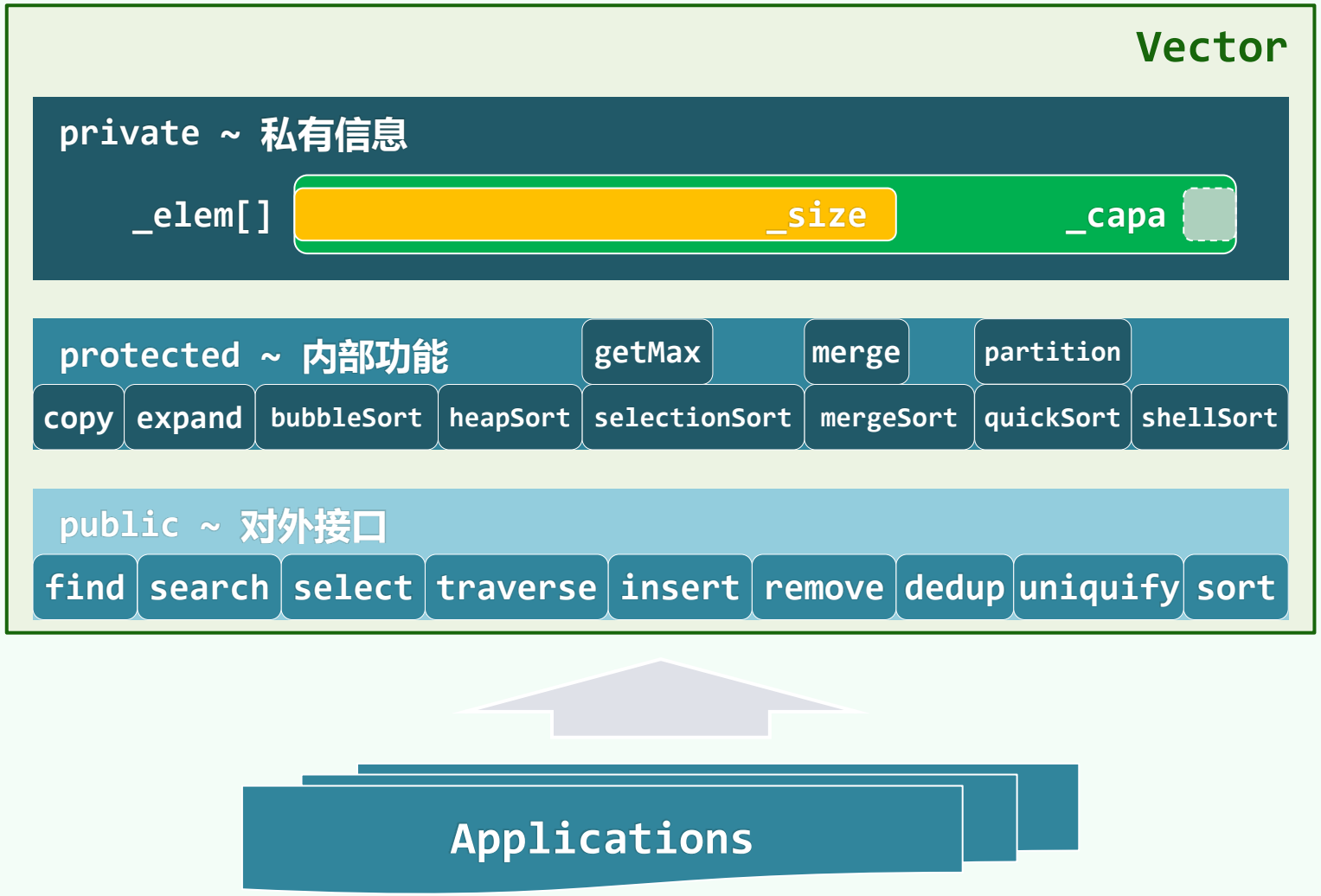
向量

ADT：模板类

邓俊辉
deng@tsinghua.edu.cn

实现者视角：Vector模板类

```
template <typename T>
class Vector { //向量模板类
   private:
      T* _elem; //数据区
      Rank _capa; //容量
      Rank _size; //规模
   protected:
      /* ... 内部接口 */
   public:
      /* ... 开放接口 */
}; //Vector
```



循秩访问

❖ 可否沿用数组的方式，便捷、直观地通过V[r]来访问向量V中的第r个元素?

可以！比如，通过重载下标操作符[]

❖ template <typename T> //可作为左值：V[r] = (T) (2*x + 3)

```
T& Vector<T>::operator[]( Rank r ) { return _elem[ r ]; }
```

❖ template <typename T> //仅限于右值：T x = V[r] + U[s] * W[t]

```
const T& Vector<T>::operator[]( Rank r ) const { return _elem[ r ]; }
```

❖ 我们将采用简易的方式处理意外和错误（比如，入口参数约定：0 <= r < _size）

构造 + 析构：重载

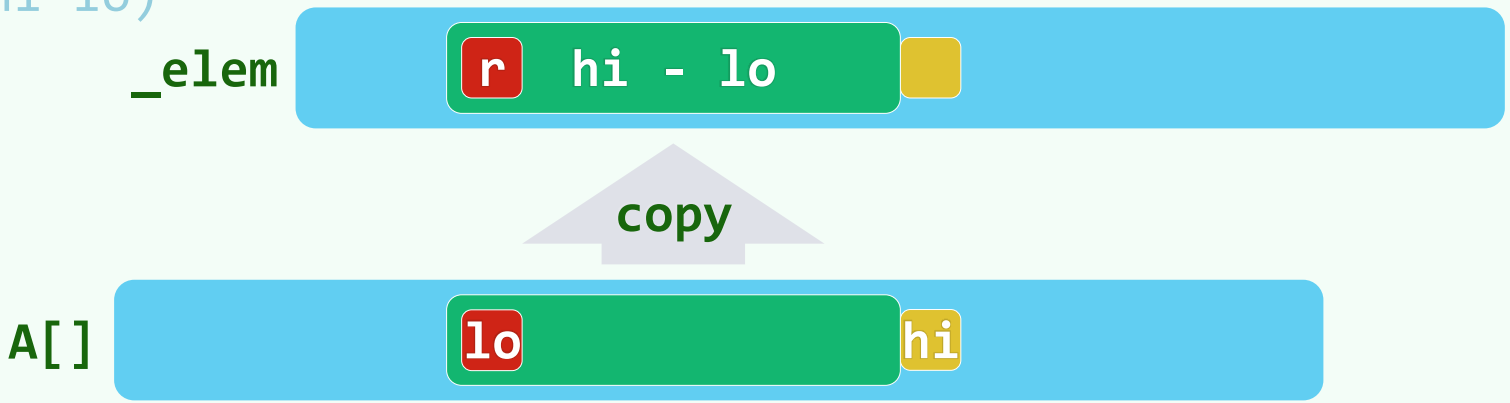
```
❖ Vector( Rank s = DEFAULT_SIZE ) //按预估规模开出足够大的空间，可免反复扩容
    { _elem = new T[ _capa = s + 1 ]; _size = 0; } //末尾总是预留一个哨兵

❖ Vector( const Vector<T>& V )
    : Vector( V._elem, 0, V._size ) {} //复制向量整体或
  Vector( const Vector<T>& V, Rank lo, Rank hi )
    : Vector( V._elem, lo, hi ) {} //向量区间或
  Vector( const T* A, Rank lo, Rank hi )
    : Vector() { insert( 0, A, lo, hi ); } //数组区间

❖ ~Vector() { delete [] _elem; } //析构所有成员对象，并释放空间
```

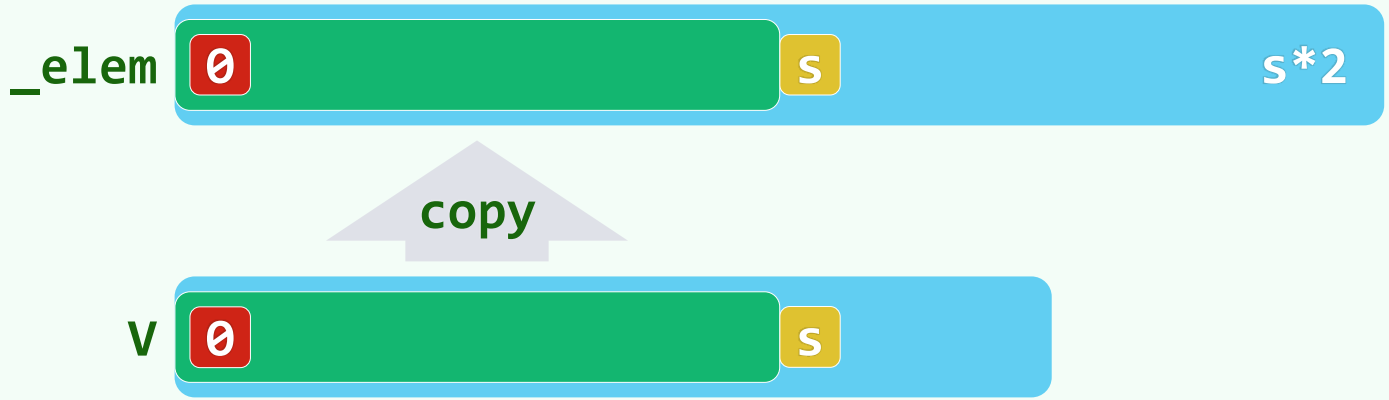
数据复制

```
template <typename T>
void Vector<T>::copy( Rank r, const T* A, Rank lo, Rank hi ) {
   while ( lo < hi ) //A[lo,hi)复制至_elem[r,r-lo+hi)；原先如有内容，直接覆盖
      _elem[ r++ ] = A[ lo++ ]; //T若非基本类型，则需重载操作符'='以实现深复制
} //copy, O(hi-lo)
```



赋值|深复制

```
template <typename T>
void Vector<T>::operator=( const Vector<T>& V ) {
   _size = 0; //清除原有内容
   expand( V._size ); //确保有足够空间
   copy( 0, V._elem, 0, _size = V._size ); //整体复制
} //operator=
```



2．向 量

02-A.抽象数据结构

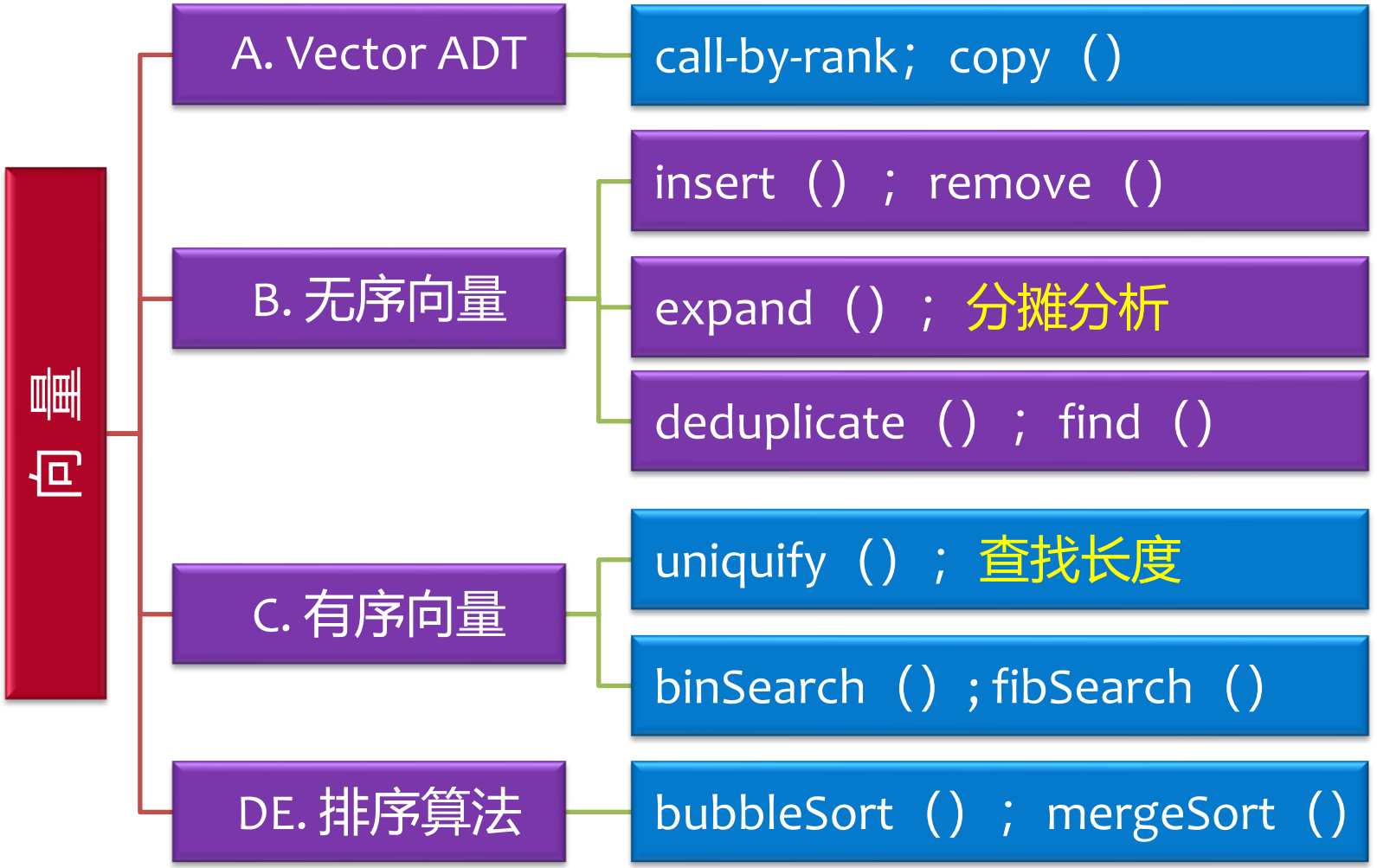
02-B.无序向量

02-C.有序向量

02-D.起泡排序

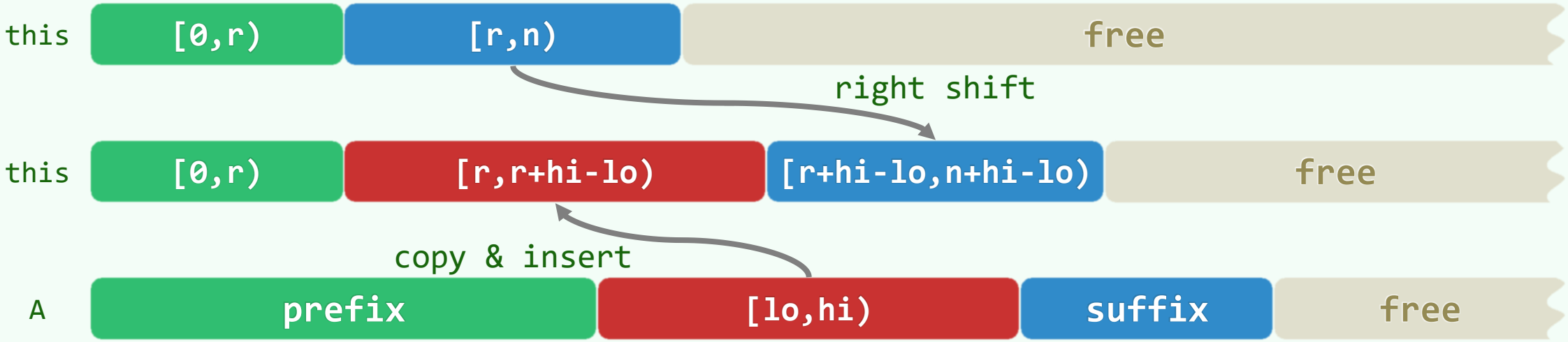
02-E.归并排序

02-F. 位图



插入：自后向前地后移

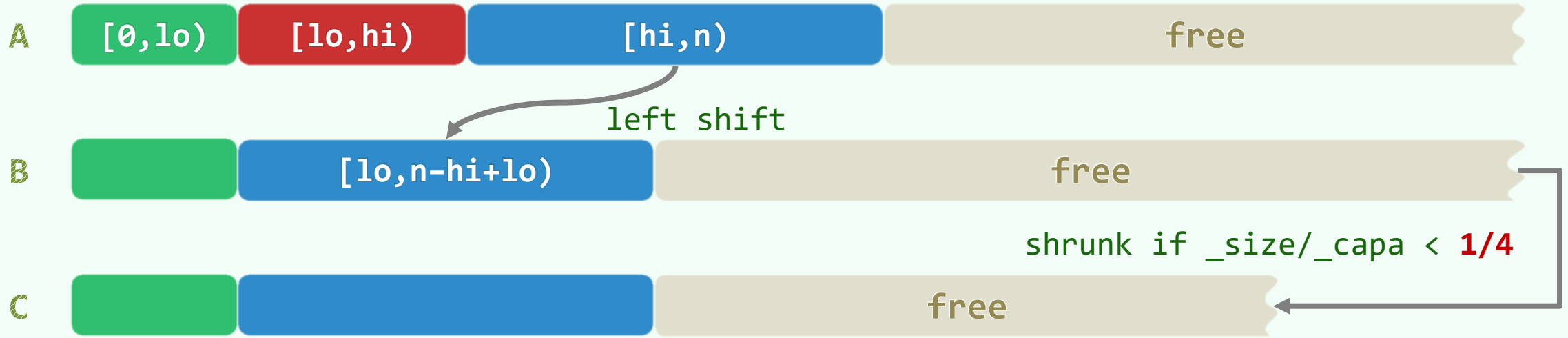
```
template <typename T> void Vector<T>::insert( Rank r, const T* A, Rank lo, Rank hi ) {
   Rank d = hi - lo;  expand( d ); //若必要，先扩容：确保足以新增d个元素
   for ( Rank i = _size-1; i != r-1; i-- ) _elem[i+d] = _elem[i]; //长度为n-r的后缀：后移
   copy( r, A, lo, hi );  _size += d; //插入d个元素
}
```




```
template <typename T> Rank Vector<T>::insert( Rank r, const T& e ) { insert(r, &e, 0, 1); }
```

删除：自前向后地前移

```
template <typename T> void Vector<T>::remove( Rank lo, Rank hi ) { //0<=lo<hi<=n
   while ( hi < _size ) _elem[ lo++ ] = _elem[ hi++ ]; //O(n-hi)：正比于后缀长度，而与hi-lo无关
   _size = lo;   shrink(); //更新规模，可能缩容
}
```




```
template <typename T> T Vector<T>::remove( Rank r )
   { T e = _elem[r];   remove( r, r+1 );   return e; } //O(n-r)，仍正比于后缀长度
```

性能分析

- 单元素的插入|删除，为何都实现为区间插入|删除的特例？ //General --> Specific

  反过来，为何不将后者视作前者的多次重复呢？ //Specific --> General

- 以remove()为例...

- G2S：后缀一次性地大跨步前移

  时间成本仅决定于后缀长度，而与删除的元素数量无关：$_\text{size} - \text{hi} = \mathcal{O}(n)$

- S2G：后缀反复地小步前移：$(_\text{size} - \text{hi}) \cdot (\text{hi} - \text{lo}) = \mathcal{O}(n^2)$

- insert()总体亦类似（尽管略有区别）

- 经验：重复性的任务应尽可能集中起来，批量地完成 //作业例外

向量

无序向量：扩容

邓俊辉
deng@tsinghua.edu.cn

空间管理

❖ 申请空间时留有足够余地，以保证在相当长的时间内无需追加空间



❖ 然而，对于任何固定的容量_capa，总是难免会出现尴尬情况...

❖ 定义：装填因子/load factor = λ = _size/_capa

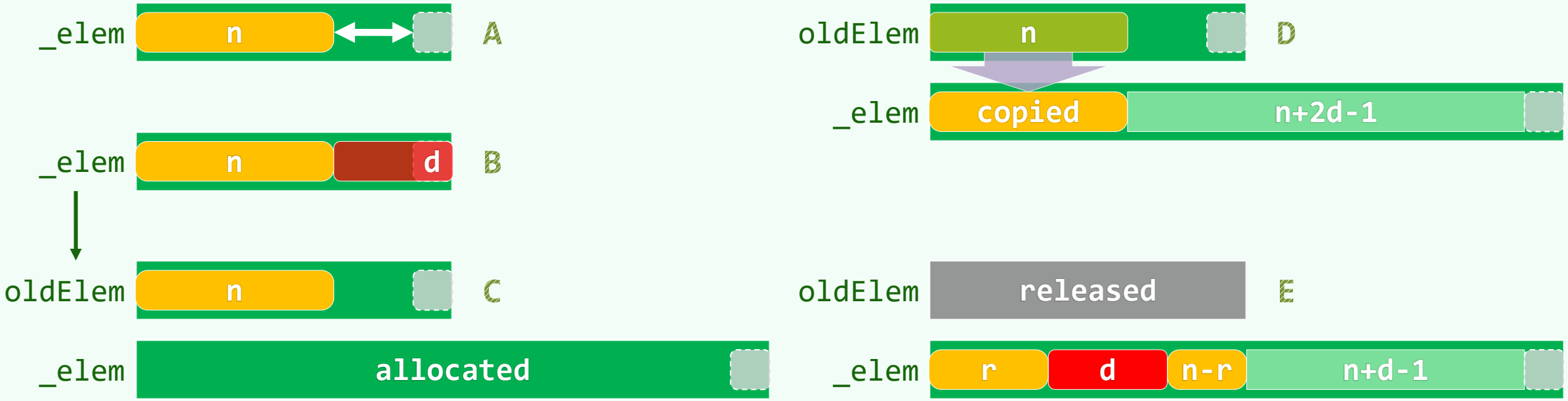
- 上溢/overflow：不足以接纳新的元素：λ = 100%
- 下溢/underflow：实际存放的元素寥寥无几：λ << 50%

❖ 为保证向量持续高效运转，需令其根据实际情况动态调整容量，始终保证：25% <= λ < 100%

动态扩容

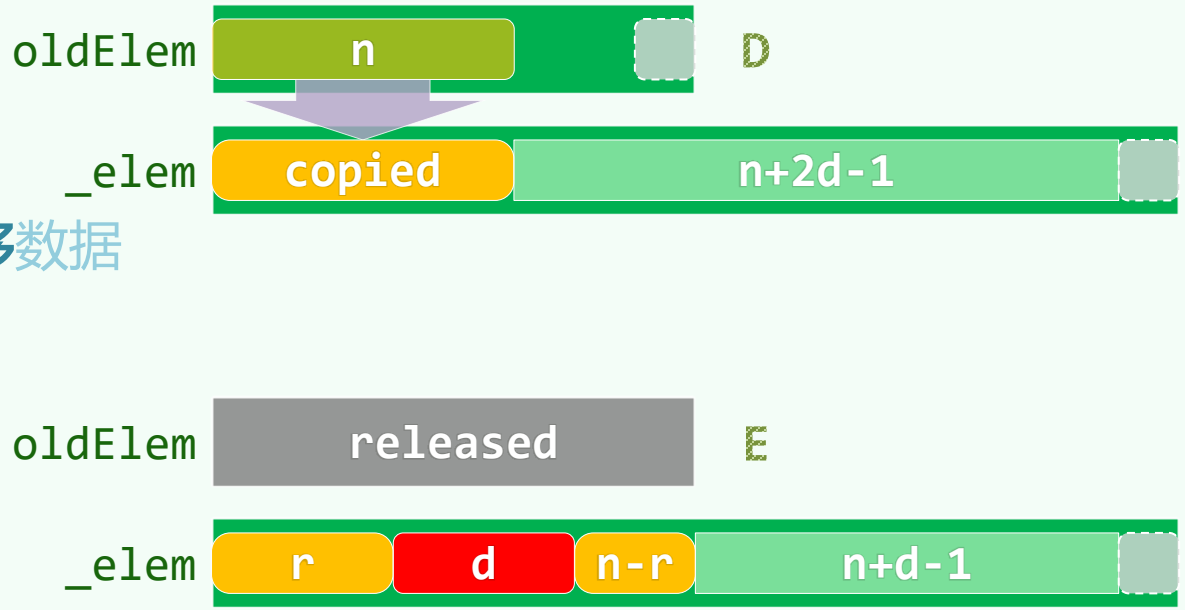
❖ 蝉：身体经过一段时间的生长，会蜕去原先的外壳，代之以更大的新外壳

❖ 向量：在即将上溢时，适当扩大内部数据区的容量



扩容算法

```
template <typename T> void Vector<T>::expand( Rank d = 1) {
    if ( _size + d < _capa ) return; //若足以新增d个元素，则无需扩容
    T* oldElem = _elem; //记下原数据区
    _capa = (_size + d) * 2; //容量加倍
    _elem = new T[ _capa ]; //分配新空间
    copy( 0, oldElem, 0, _size ); //转移数据
    delete [] oldElem; //释放原空间
} //为何采用了容量加倍策略?
```



- 最好O(1)，最坏O(n)；相差悬殊，且非此即彼

  ——如何从整体上界定其性能呢?

向量

无序向量：均摊分析

邓俊辉
deng@tsinghua.edu.cn

平均分析?

❖ 平均复杂度|average complexity ~ 期望复杂度|expected complexity

- 根据各种操作出现的概率，对其运行时间做加权平均

❖ 很遗憾，平均分析法并不适用于expand()，因为...

- 其最好、最坏情况的出现并非随机的

❖ 对任何一个真实的向量，考查其完整的、足够长的历史

（简化起见，设仅有插入而无删除，且都是单元素插入）

- 截至_size = n >> 2时，必已做过n次insert()，进而expand()也被调用过n次

- 纵观这n次调用，它们其实是完全确定的...

均摊分析！

❖ ...

- 每出现一次O(n)的情况，_capa总会加倍，随后的装填因子总是50%
- 于是，接下来需要至少再做n次插入，才会再次因上溢而出现O(n)的情况
- 而此前对应的n次expand()调用必然都属于O(1)的情况

❖ 这样的一个向量对expand()的连续调用，累积的时间成本总是决定于初始状态，以及调用次数

❖ 均摊复杂度|amortized complexity

- 考查对一个真实算法连续的、足够多次的调用，将其间总共花费的时间，摊还至单次调用

如此，方能更加精准、更加理性地评判一个数据结构和算法的真实性能

容量加倍策略

```
_capa = (_size + d) * 2;
```

- 假设：从初始容量为1开始

  逐个地插入 $n = 2^m - 1$ 个元素（无删除）

- 最好情况不超过n次，累计耗时 $\mathcal{O}(n)$

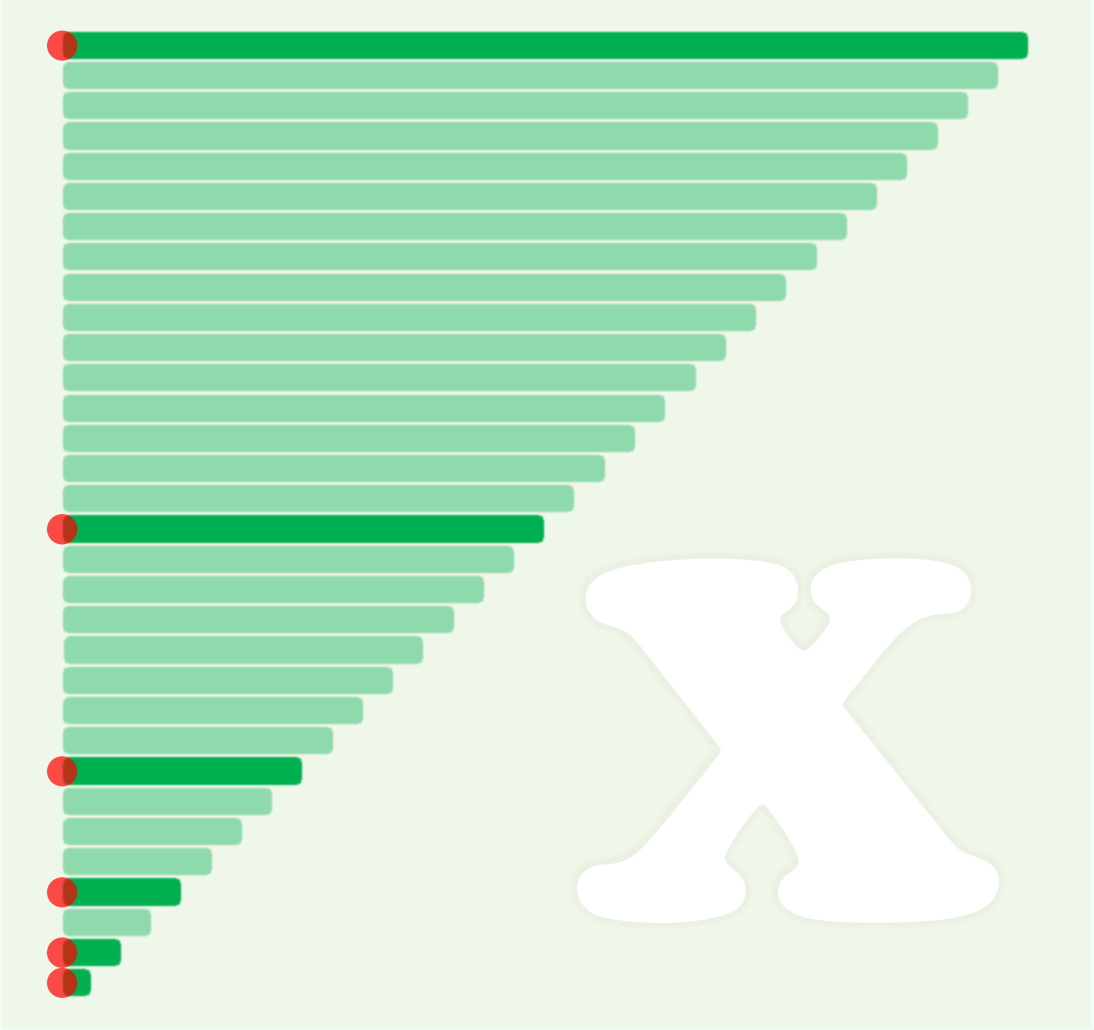
- 最坏情况只出现在第1,2,4,8,16,32,...次插入

  （分别扩容至_capa = 2,4,8,16,32,64,...）

- 时间主要消耗于数据的复制搬迁

  构成一个以2为倍数的几何级数，累计 $\mathcal{O}(n)$

  摊还至每一次expand()调用，只有 $\mathcal{O}(1)$ ！



容量递增策略 `_capa += INCREMENT;`

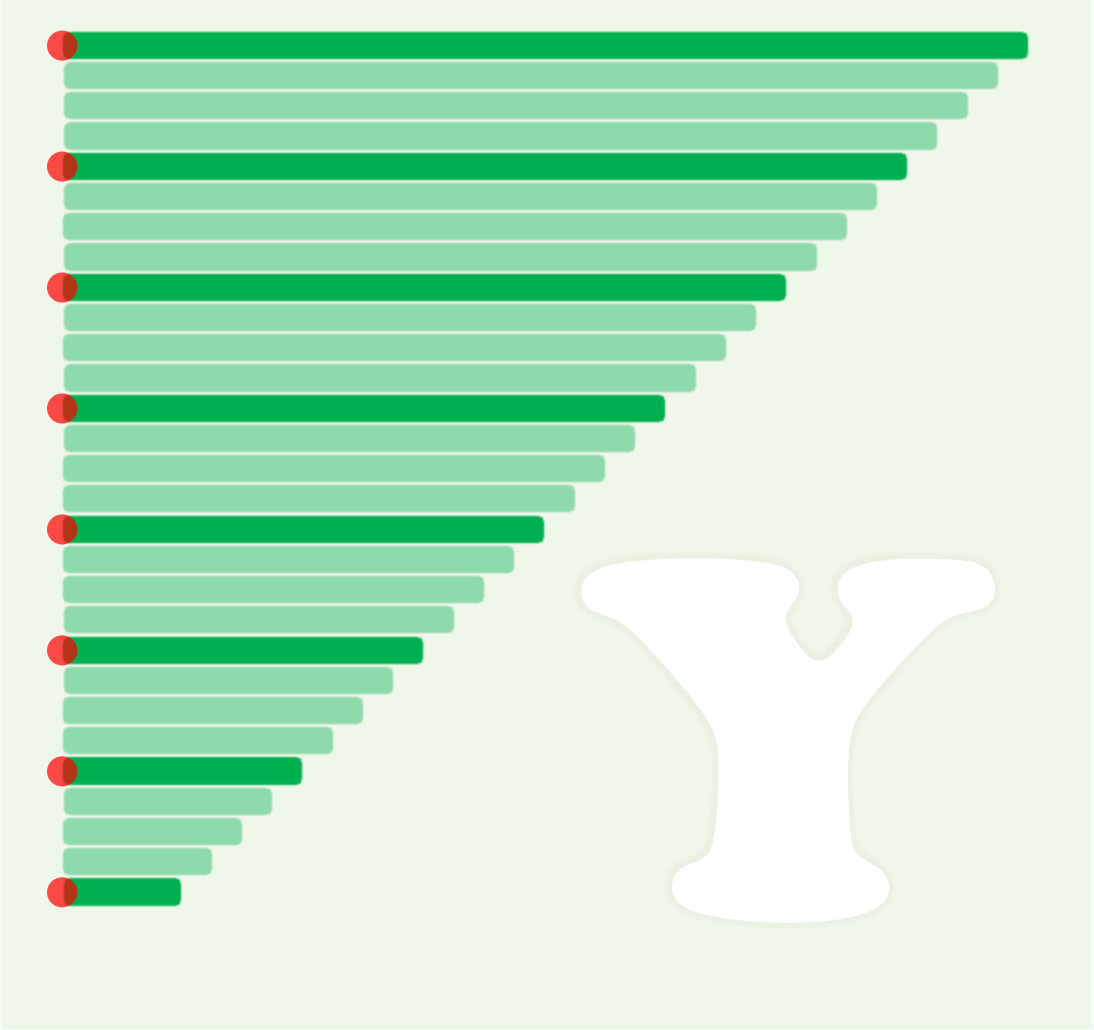
- 假设从初始容量I开始，渐次地按增量I扩容
  - 忽略最好情况
  - 每隔I次调用，便会出现一次最坏情况

- 时间依然主要消耗于数据的复制搬迁

  构成一个以I为增量的算术级数，累计

$$\Omega(n^2/I/2) = \Omega(n^2)$$

  摊还至每一次expand()调用，高达 $\Omega(n)$ ！



向量

无序向量：查找与去重

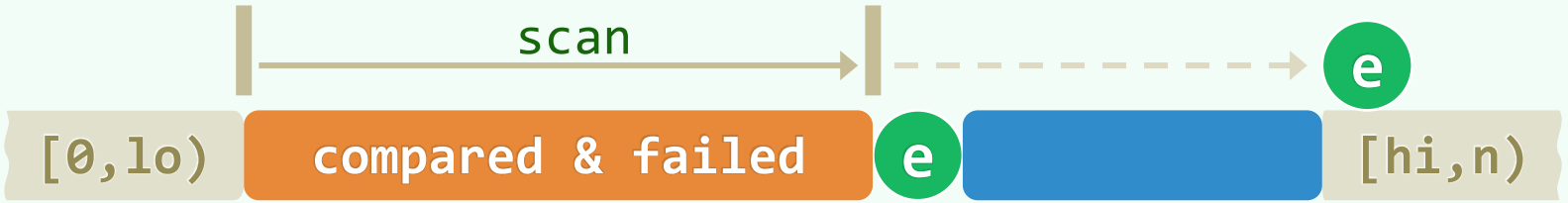
邓俊辉
deng@tsinghua.edu.cn

判等器 ~ 比较器

```
template <typename K, typename V> struct Entry { //词条
   K key; V value; //键（亦称关键码）、值
   Entry( K k = K(), V v = V() ) : key ( k ), value ( v ) {}; //默认构造函数
   Entry( const Entry<K, V>& e ) : key ( e.key ), value ( e.value ) {}; //克隆
// 被判为相等的词条，未必相同
   bool operator== ( const Entry<K, V>& e ) { return key == e.key; }  //等于
   bool operator!= ( const Entry<K, V>& e ) { return key != e.key; }  //不等于
// 得益于比较器和判等器，从此往后，不必严格区分词条及其对应的键
   bool operator<  ( const Entry<K, V>& e ) { return key <  e.key; }  //小于
   bool operator>  ( const Entry<K, V>& e ) { return key >  e.key; }  //大于
}; //Entry
```

顺序查找

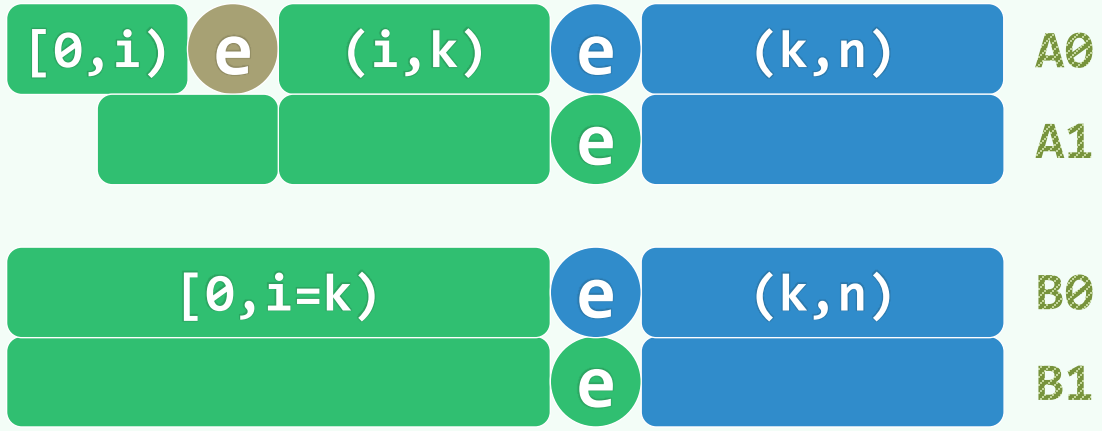
```
❖ template <typename T> //0 <= lo < hi <= _size
  Rank Vector<T>::find( const T& e, Rank lo, Rank hi ) const { //O(hi-lo)
     T bak = _elem[hi]; _elem[hi] = e; //先将目标存入哨兵
     while ( e != _elem[lo] ) lo++; //再从前向后，逐个比对
     _elem[hi] = bak; return lo; //哨兵还原；返回最靠前的命中者，或hi示意失败
  } //find
```



❖ 输入敏感|input-sensitive：最好O(1)，最差O(n)，有实质区别

去重：减而治之

```
❖ template <typename T> Rank Vector<T>::dedup() { //剔除相等的元素，保留最靠后者
    for ( Rank k = 0; k < _size;  ) {
       Rank i = find( _elem[k], 0, k ); //O(i+1)
       if ( i < k )
          remove(i); //O(n-i-1)
       else
          k++; //O(1)
  } //O(n^2)
```



❖ [k]就在哨兵位置，find()本来不必复原设置哨兵——好在，不伤大雅

❖ 主要的操作无非元素的比对与移动：第04章将通过均摊分析确定，其总数无论如何总是n(n+1)/2

向量

无序向量：遍历

邓俊辉
deng@tsinghua.edu.cn

遍历：对向量中的每一个元素，实施统一的操作

- 如果遍历是列队操演

  那么统一的操作就是号令：

  - 立正、稍息、向右看齐、报数...

- 号令如何描述？如何传达至每个元素？

- 函数对象|Function Object

  - 本是一个对象 //visit
  - 但在重载了“()”之后

    等效于一个函数 //visit()

```
template <typename T>

template <typename VST>

      XX排XX班          全体都有          向前一步走
        ↓                 ↓                 ↓
void Vector<T>::traverse( VST& visit ) {

   for ( Rank i = 0; i < _size; i++ )

      visit( _elem[i] );

}
```

单兵动作：统一地将向量中的元素各自加一

❖ 先实现一个类，重载圆括号，可使单个T类型的元素加一

```
template <typename T> //假设T可直接递增或已重载操作符“++”
struct Increase //函数对象：通过重载操作符“()”实现
   { void operator()( T& e ) { e++; } }; //加一
```

❖ 再将其作为参数传递给遍历算法，即可以之作为基本操作来遍历向量

```
template <typename T> void increase( Vector<T>& V )
   { V.traverse( Increase<T>() ); }
```

❖ 课后练习：模仿此例，实现统一的减一、加倍等更多的遍历功能

整体动作：统计元素总和，作为校验值

```
❖ template <typename T> struct Cumulate { //累计一个T类型对象的数值
      T& s;
      Cumulate( T& sum ) : s( sum ) {}
      void operator() ( T& e ) { s += e; } //约定T可直接相加
  }; //Cumulate

❖ template <typename T> T sum( Vector<T>& V ) { //统计向量中所有元素的总和
      T sum = 0;
      V.traverse( Cumulate<T>( sum ) ); //以Cumulate为基本操作进行遍历
      return sum; //返回总和
  } //sum
```

复杂整体动作：统计逆序对总数，判断是否有序

```
❖ template <typename T> struct Compare { //判断线性序列中的一个元素是否与其前驱顺序
     T pred; int& i; //前驱、（相邻）逆序对的计数器
     Compare( int& invs, T& first ) : pred( first ), i( invs ) {}
     void operator() ( T& e ) { i += ( pred > e ); pred = e; }
  }; //Compare

❖ template <typename T> int inv( Vector<T>& V ) { //统计向量中相邻逆序对的总数
     int invs = 0;
     V.traverse( Compare<T>( invs, V[0] ) ); //以Compare为基本操作做遍历
     return invs; //返回逆序对总数：为零当且仅当向量整体有序
  } //inv
```

2．向 量

02-A.抽象数据结构

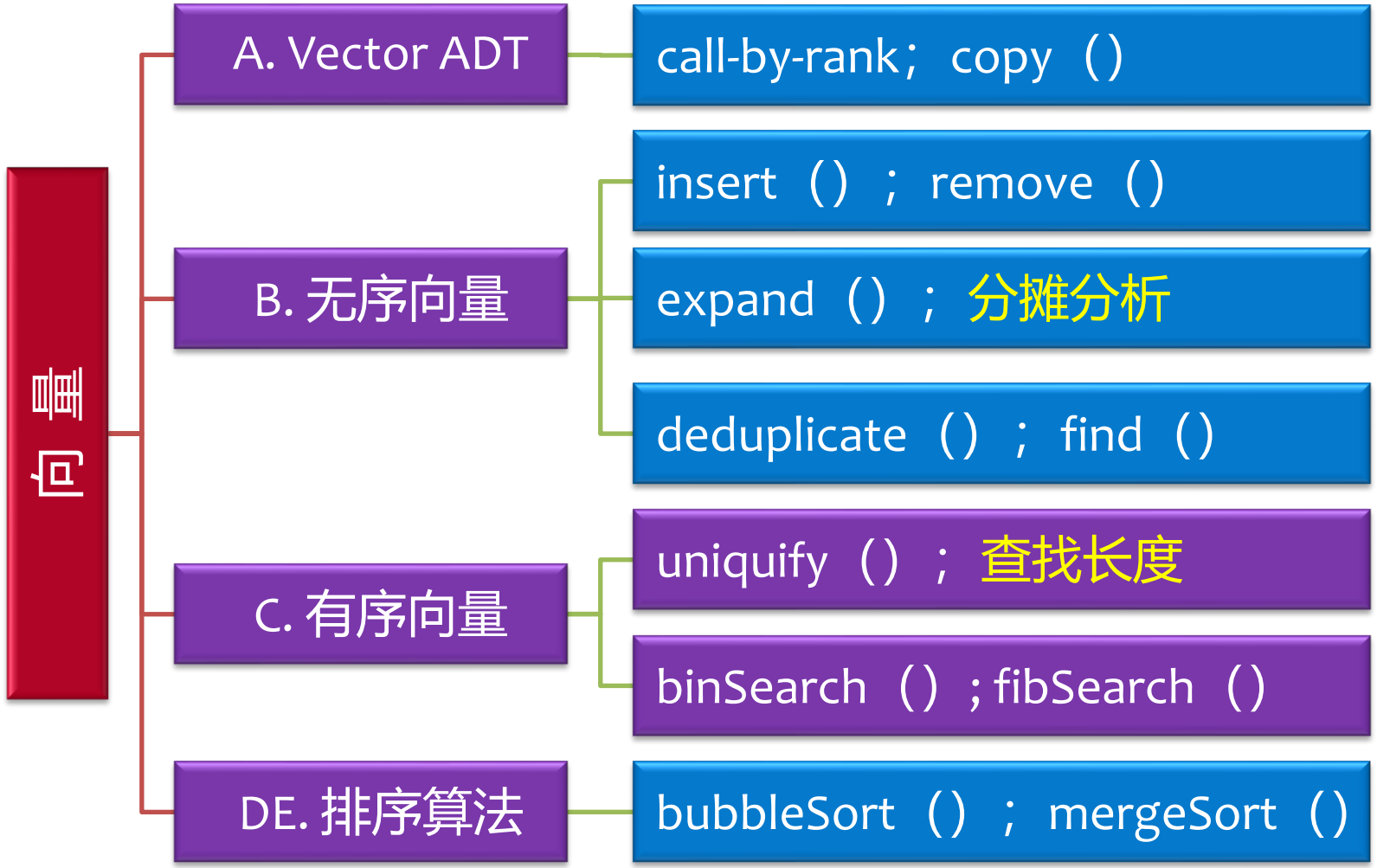
02-B.无序向量

02-C.有序向量

02-D.起泡排序

02-E.归并排序

02-F. 位图



02.Vector

C-有序向量

向量

有序向量：去重

邓俊辉
deng@tsinghua.edu.cn

勤奋的低效算法

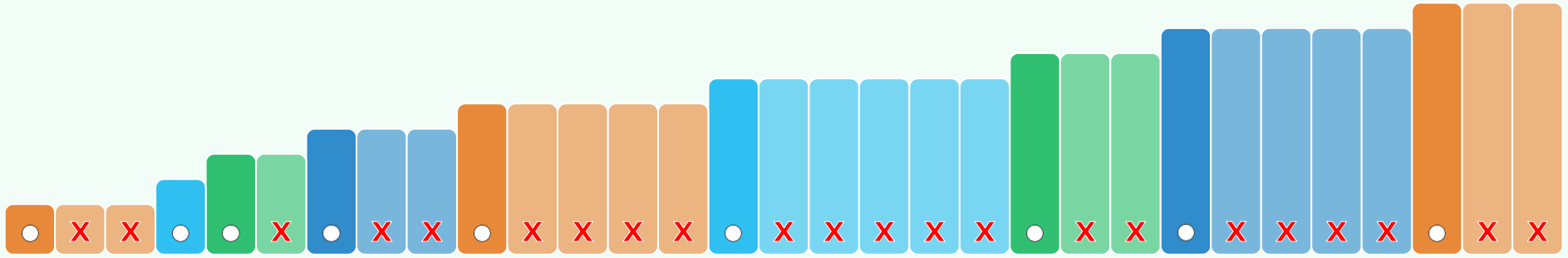
❖ 观察：有序向量中的每一组重复元素，都会彼此聚集地占据一个区段

约定：每个区段仅保留第一个元素

❖
```
template <typename T> void Vector<T>::uniquify() {
    for ( Rank i = 1; i < _size; ) //从前向后，逐一比对相邻的[i-1]和[i]
        if ( _elem[i-1] != _elem[i] ) i++; //若不等，则转至下一对元素
        else remove( i ) : //否则，删除居后者
} //O(n^2)
```

❖ 耗时的remove()

调用太多！



低效率分析

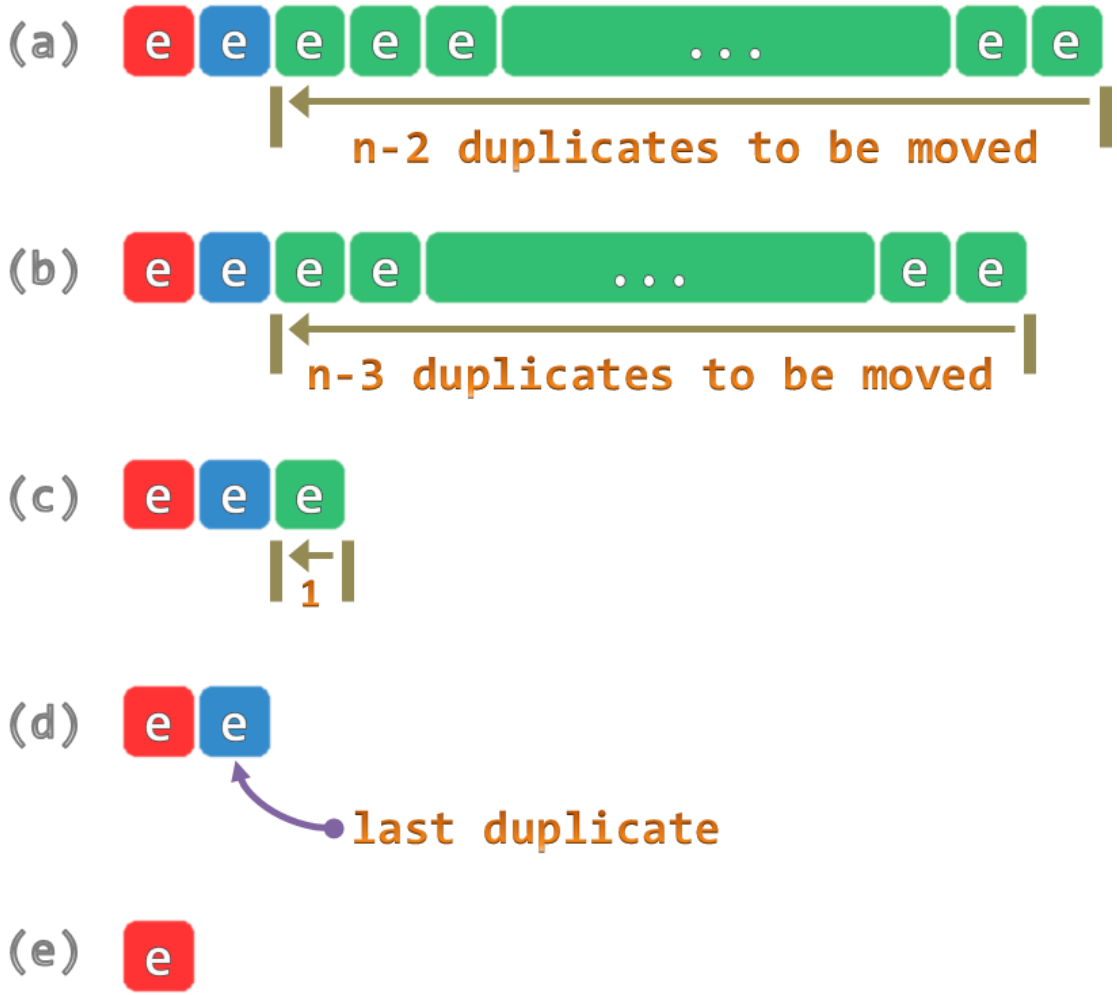
- 运行时间主要取决于while循环次数

  共计：_size - 1 = n - 1

- 最坏情况下，每次都需调用remove()
  - 各耗时O(n-1) ~ O(1)；累计$O(n^2)$
  - 尽管省去find()，总体竟与

    无序向量的deduplicate()相同

- 低效的根源在于
  - 同一元素可作为被删除元素的后继
  - 多次被前移

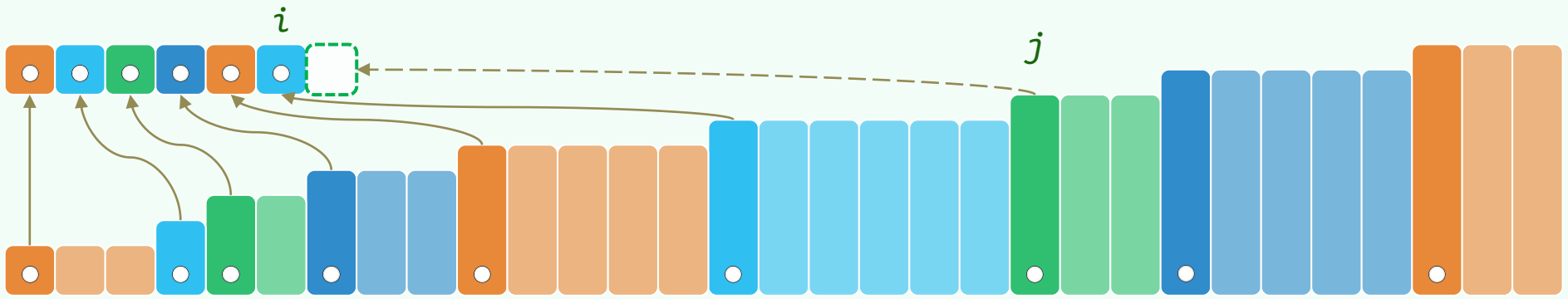


懒惰的高效算法：two-pointer technique

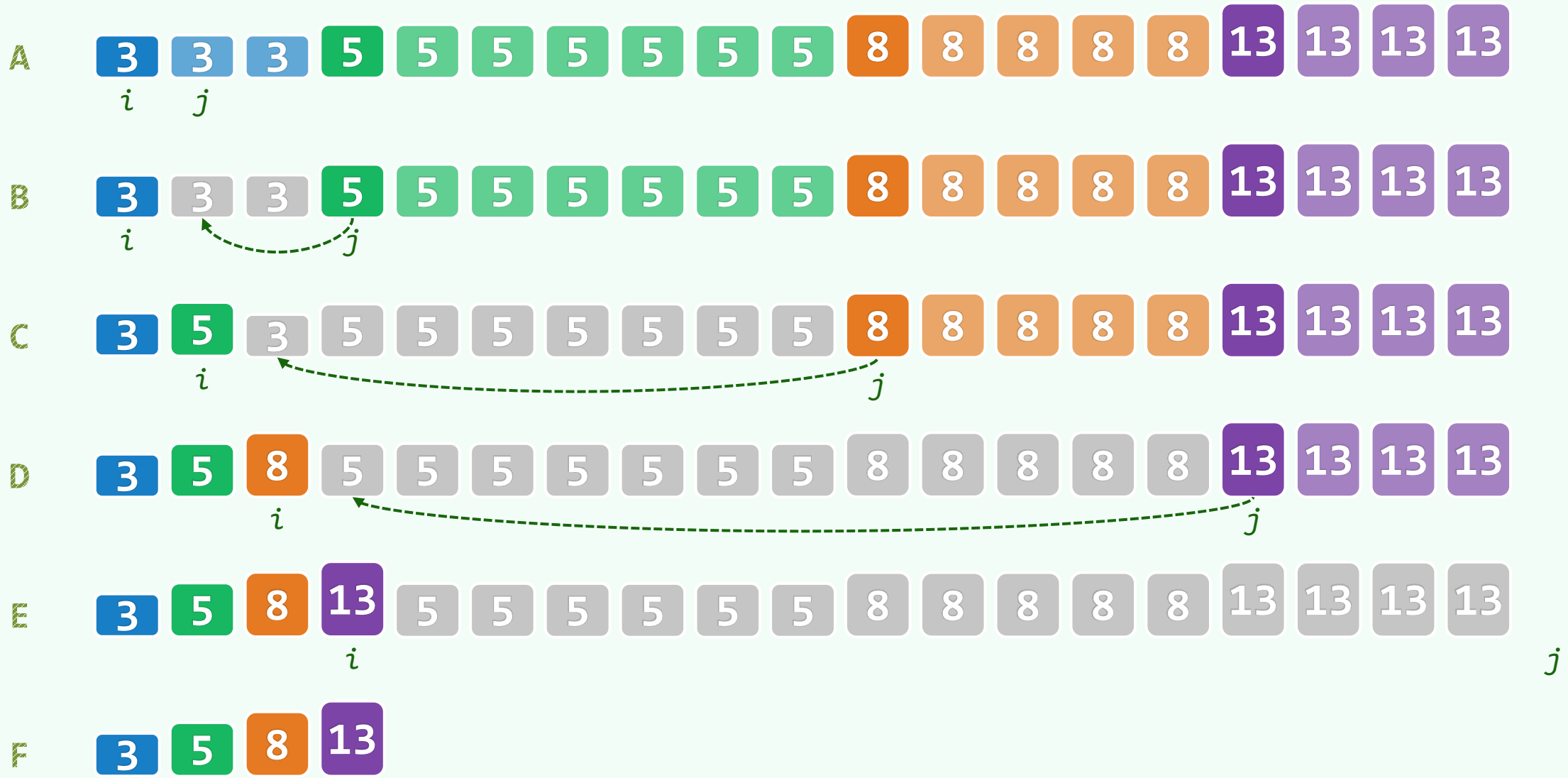
❖ 调转视角，去重的效果也可等效地理解为，仅保留各区段的首元素

❖
```
template <typename T> void Vector<T>::uniquify() {
    Rank i = 0;  //前缀[0,i]包含目前已知的所有无重元素
    for ( Rank j = 1; j < _size; j++ ) //逐一考查[j]
       if ( _elem[ i ] != _elem[ j ] ) //不等，当且仅当[j]为一个区间的首元素
          _elem[ ++i ] = _elem[ j ]; //故将[j]归入无重的前缀
    _size = ++i;
    shrink();
} //O(n)
```

❖ 每一步仅$O(1)$



实例


A
3 3 3 5 5 5 5 5 5 5 8 8 8 8 8 13 13 13 13
i j
B
3 3 3 5 5 5 5 5 5 5 8 8 8 8 8 13 13 13 13
i j
C
3 5 3 5 5 5 5 5 5 5 8 8 8 8 8 13 13 13 13
i j
D
3 5 8 5 5 5 5 5 5 5 8 8 8 8 8 13 13 13 13
i j
E
3 5 8 13 5 5 5 5 5 5 8 8 8 8 8 13 13 13 13
i j
F
3 5 8 13

向量

有序向量：二分查找（版本A）

邓俊辉
deng@tsinghua.edu.cn

接口与算法

```
❖ template <typename T> //整体查找
  Rank search( const T& e ) const { return search( e, 0, _size ); }
```




```
❖ template <typename T> //区间查找：0 <= lo < hi <= _size
  Rank Vector<T>::search( const T& e, Rank lo, Rank hi ) const {
     return ( rand() % 2 ) ? //为便于测试，暂且随机地调用
        binSearch( _elem, e, lo, hi ) : //二分查找算法，或
        fibSearch( _elem, e, lo, hi ) ; //Fibonacci查找算法
  } //在实际应用中，你可针对具体问题选用最适宜的一种
```

轴点 ~ 减治

❖ 有序向量中，每个元素都是轴点：$[lo, mi) \leq [mi] \leq (mi, hi)$，因此...

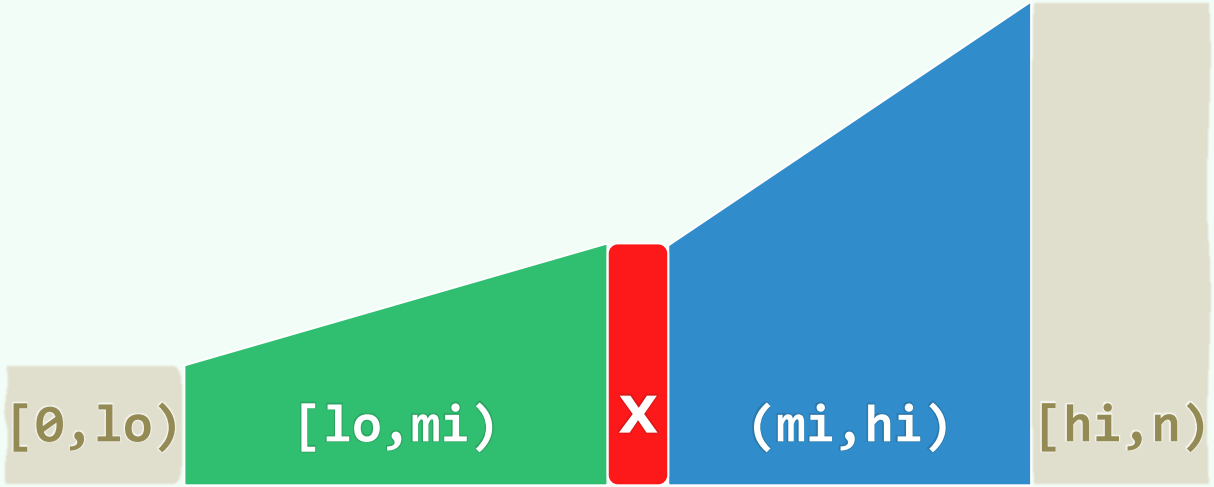
❖ 视目标元素e与[mi]的相对大小，无非三种情况：

- e < [mi]：e只能存在于前缀中，故可减除[mi,hi)并递归深入[lo,mi)
- [mi] < e：e只能存在于后缀中，亦可减除[lo,mi]并递归深入(mi,hi)
- e = [mi]：命中，查找结束

❖ 若[mi]总是取作中位数，则每递归一次

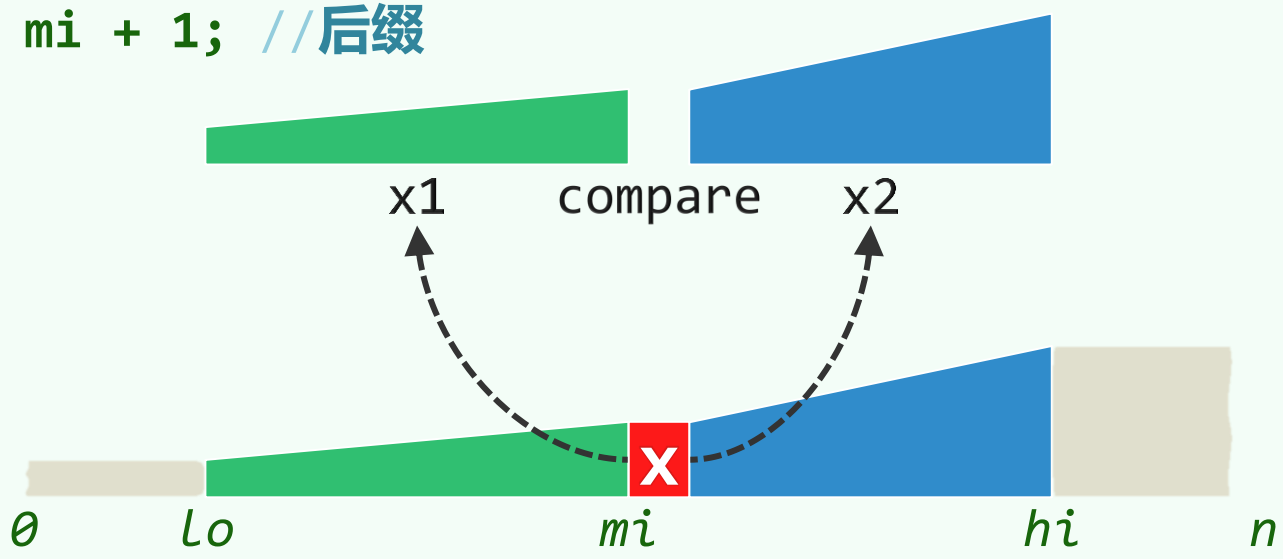
- 要么问题规模减半
- 要么能够命中

递归深度$\mathcal{O}$(logn)



实现

```
template <typename T> Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   while ( lo < hi ) { //每步迭代三个分支，最多两次比较
      Rank mi = (lo + hi) / 2; //以中位数为轴点，区间宽度折半
      if ( e < S[mi] ) hi = mi; //深入前缀或
      else if ( S[mi] < e ) lo = mi + 1; //后缀
      else return mi; //命中
   } //出口时lo==hi
   return -1; //失败
} //但-1不能指示失败的位置
```



算法举例（版本A）：Binsearch(S,77,0,7)

0	1	2	3	4	5	6
22	33	55	77A	77B	77C	88

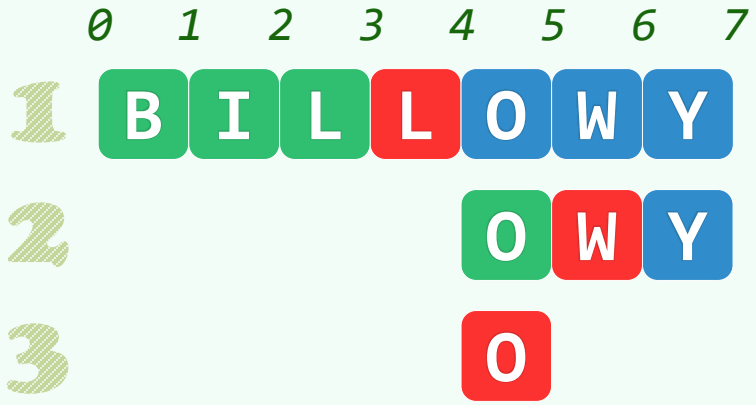
```
template <typename T> Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   while ( lo < hi ) { //每步迭代三个分支，最多两次比较
      Rank mi = (lo + hi) / 2; //以中位数为轴点，区间宽度折半
      if ( e < S[mi] ) hi = mi; //深入前缀[lo, mi)
      else if ( S[mi] < e ) lo = mi + 1; //深入后缀(mi, hi)
      else return mi; //命中，收工：有多个e时，不能保证返回秩最大者
   } //出口时lo==hi
   return -1; //失败
} //但-1不能指示失败的位置
```

lo	mi	hi
0	3	7
版本A		

实例 + 复杂度

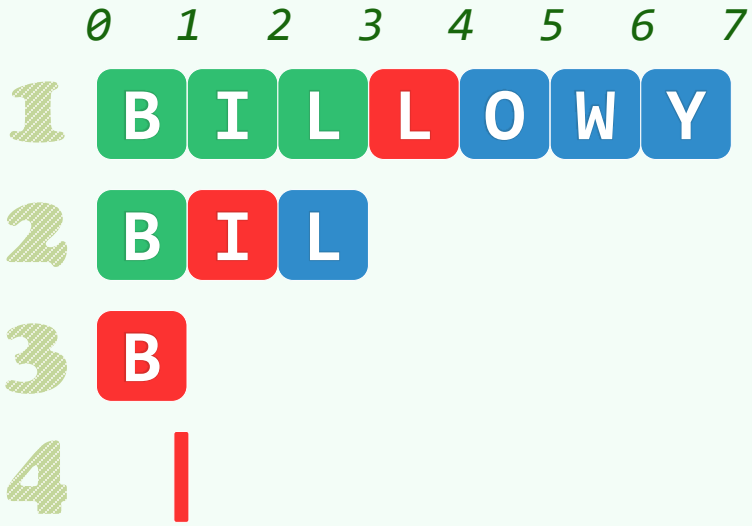
binSearch('O')

经2+1+2=5次比较，命中于[4]



binSearch('F')

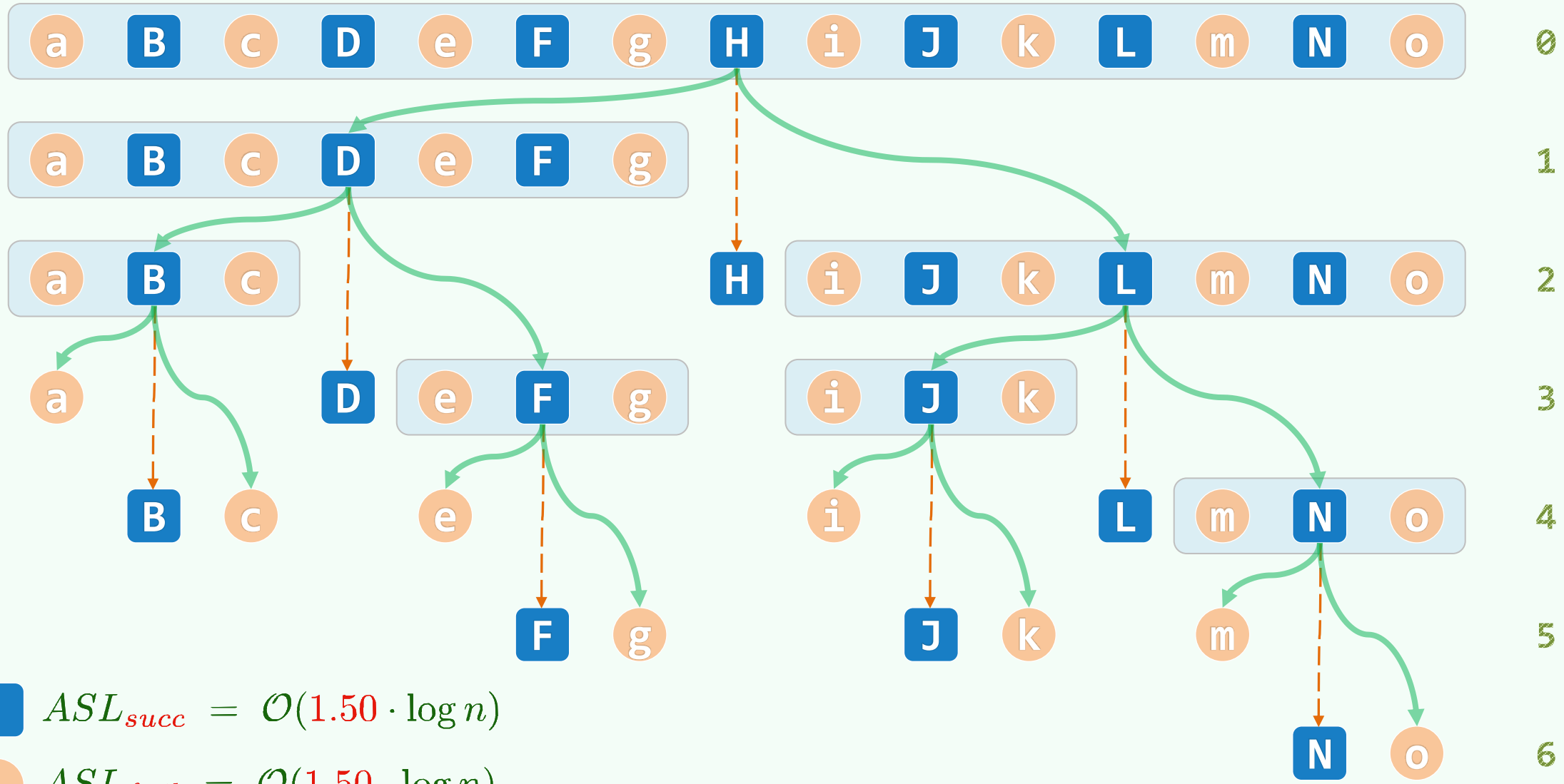
经1+1+2=4次比较，失败于[1]



❖ **线性递归：** $T(n) = T(n/2) + \mathcal{O}(1) = \mathcal{O}(\log n)$，**大大优于顺序查找**

递归跟踪：深度 $\mathcal{O}(\log n)$ **；各递归实例仅耗时** $\mathcal{O}(1)$

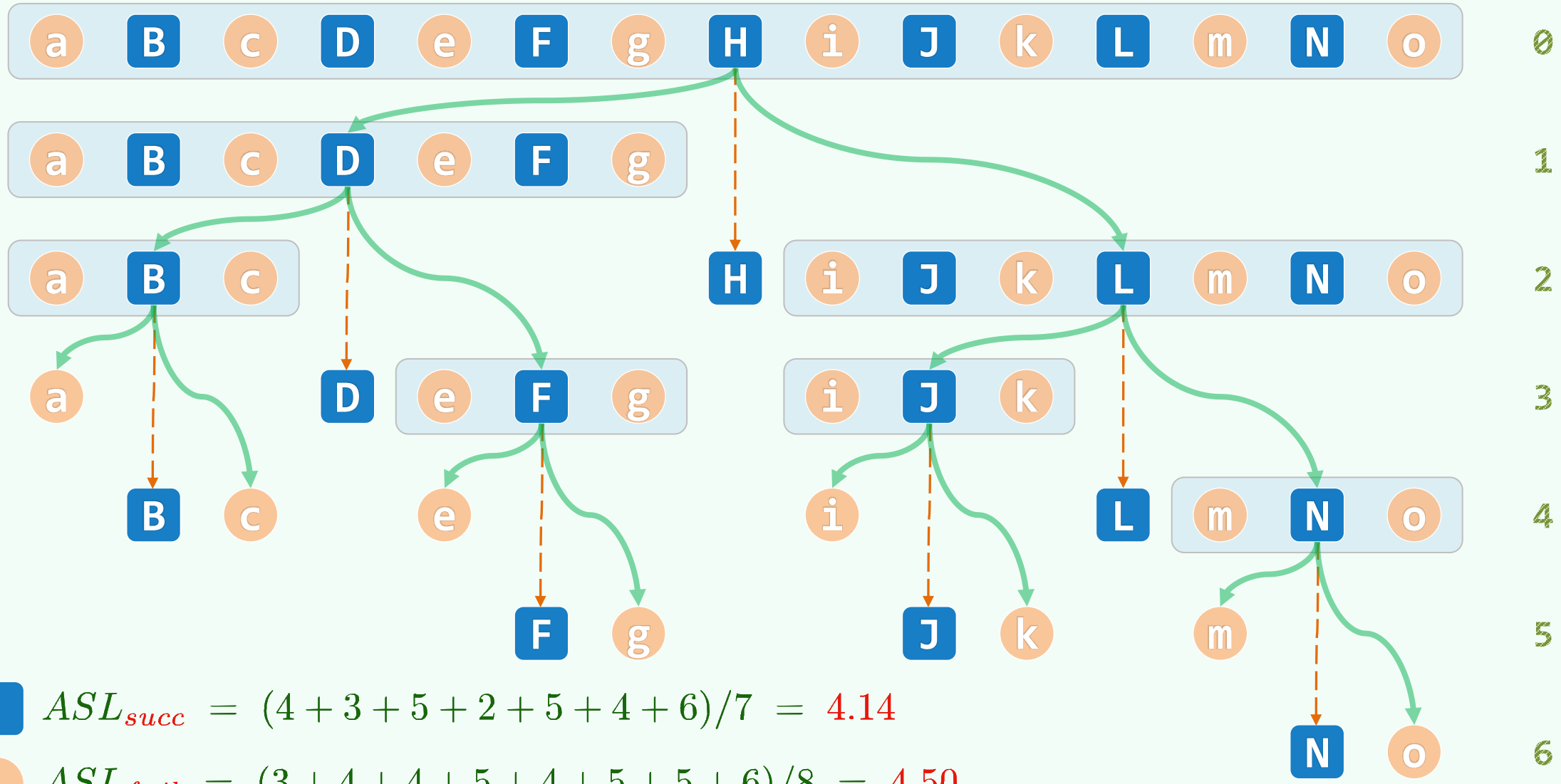
元素比较的次数 ~ 平均查找长度|average search length



$ASL_{succ} = \mathcal{O}(1.50 \cdot \log n)$

$ASL_{fail} = \mathcal{O}(1.50 \cdot \log n)$

实例：n = 7，按等概率估算



$ASL_{succ} = (4+3+5+2+5+4+6)/7 = 4.14$

$ASL_{fail} = (3+4+4+5+4+5+5+6)/8 = 4.50$

向量

有序向量：二分查找（版本B）

邓俊辉
deng@tsinghua.edu.cn

改进思路

❖ 事实：查找成功的概率通常极低：对于 $n = 10^6$ 个uint32_t组成的有序向量，仅 $10^6/2^{32} < 1/4000$

悖论：为了如此难得一现的情况，此前的算法居然在每一步迭代中都花费时间做了一次比较！

❖ 启示：与其贪恋虚妄、踟蹰流连，不如加快步伐、埋头向前

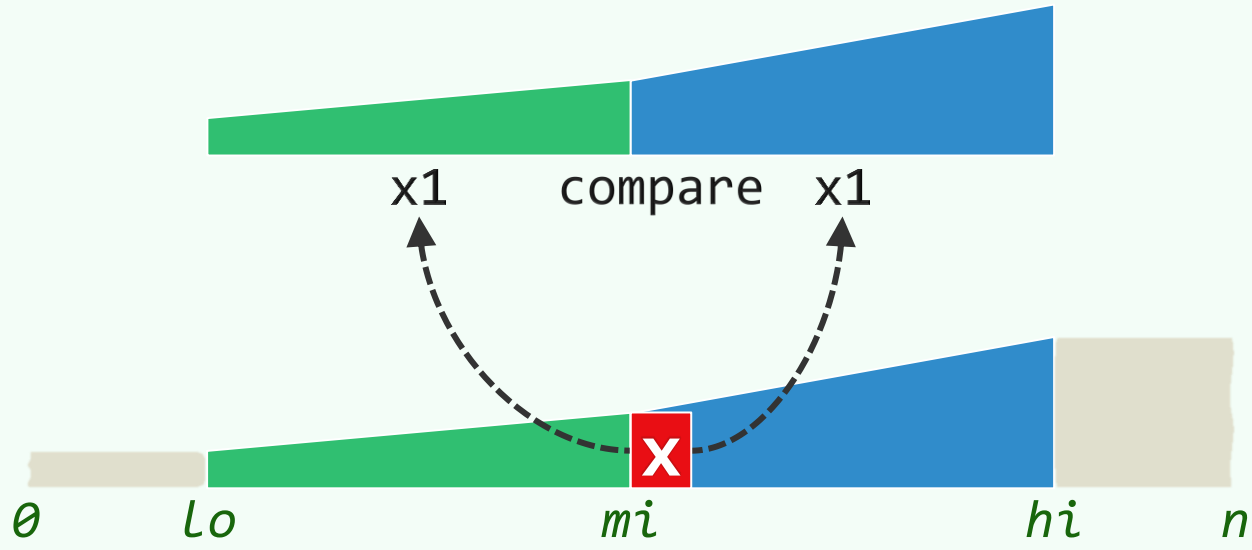
行动：每次迭代仅做1次元素比较，只有2个分支方向（放弃“可能”命中的分支）

- e < x: 则深入左侧的[lo, mi)
- x <= e: 则深入右侧的[mi, hi)

❖ 持续收缩范围，直到hi-lo = 1

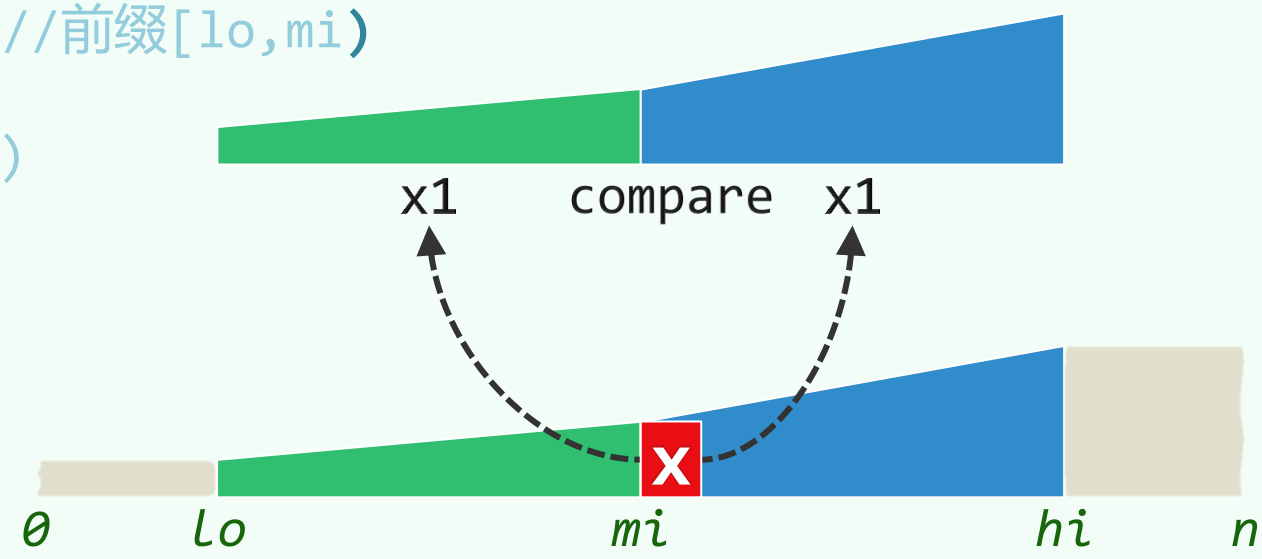
❖ 相对于版本A，最好|坏情况下更坏|好

整体性能有所提高



实现

```
template <typename T> Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   if ( e < S[lo] ) return lo - 1; //相当于e = -inf
   while ( 1 < hi - lo ) { //每步迭代一次比较、两个分支；不能提前终止
      Rank mi = (lo + hi) / 2; //轴点
      if ( e < S[mi] ) hi = mi; //前缀[lo,mi)
      else lo = mi; //后缀[mi,hi)
   } //出口时hi = lo+1
   return lo;
} //binSearch
```



算法举例（版本B）：Binsearch(S,77,0,7)

0	1	2	3	4	5	6
22	33	55	77A	77B	77C	88

```
template <typename T> Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   if ( e < S[lo] ) return lo - 1; //相当于e = -inf
   while ( 1 < hi - lo ) { //每步迭代一次比较、两个分支；不能提前终止
      Rank mi = (lo + hi) / 2; //以中位数为轴点，区间宽度折半
      if ( e < S[mi] ) hi = mi; //经比较后确定深入前缀[lo,mi)
      else lo = mi; //或后缀[mi,hi)
   } //出口时hi = lo+1
   return lo;
} //binSearch
```

lo	mi	hi
0	3	7
3	5	
5	6	6
版本B		

返回值

❖ binSearch()的版本B，总是返回不大于e的最大者：$m = \max(\{-1\} \cup \{\, k \mid S[k] \le e \,\})$

❖ 得益于此，有序向量才可以便捷地维护

```
V.insert( 1 + V.search(e), e )
```

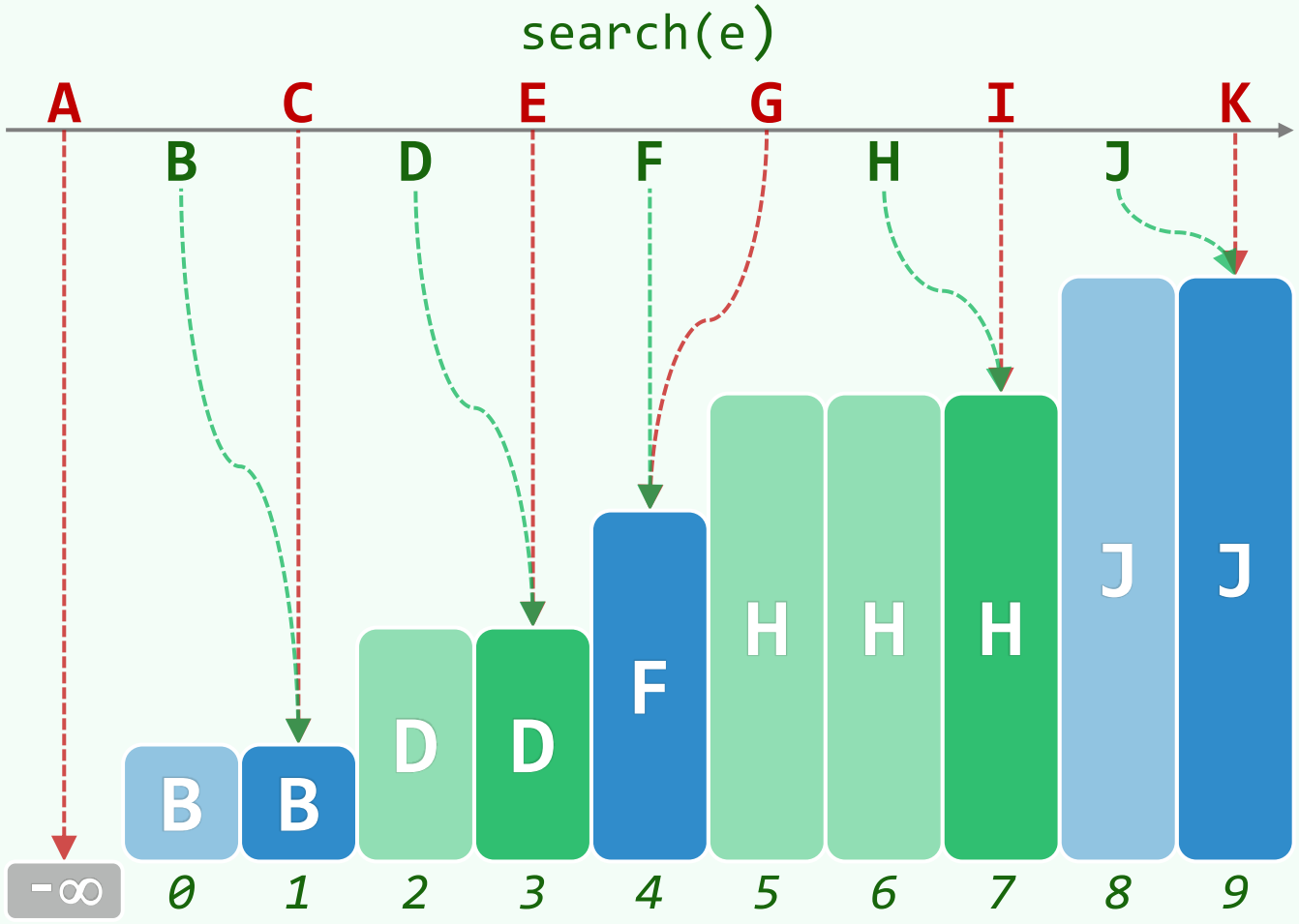
❖ 有序性：即便失败，也给出e可安身之处

稳定性：相等的元素按插入次序排列

先到者居前，晚来者靠后

❖ 然而，e<[lo]的情况仍需特判

同时，ASL还可进一步优化...



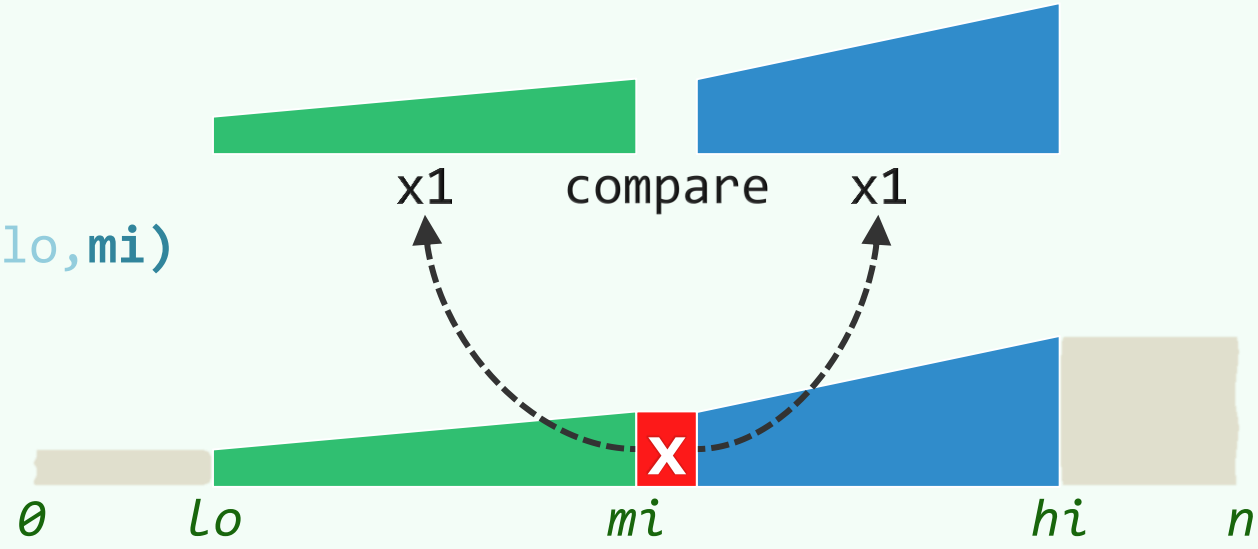
向量

有序向量：二分查找（版本C）

邓俊辉
deng@tsinghua.edu.cn

实现

```
template <typename T> Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   while ( lo < hi ) { //不变性：[0,lo) <= e < [hi,n)
      Rank mi = (lo + hi) / 2;
      if ( e < S[mi] ) hi = mi; //[lo,mi)
      else lo = mi + 1; //(mi,hi)
   } //出口时，[lo = hi] = M
   return lo - 1; //至此，[lo]为大于e的最小者，故[lo-1] = m即为不大于e的最大者
} //binSearch
```



算法举例（版本C）：Binsearch(S,77,0,7)

0	1	2	3	4	5	6
22	33	55	77A	77B	77C	88

```
template <typename T> Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   while ( lo < hi ) { //不变性：[0, lo) <= e < [hi, n)
      Rank mi = (lo + hi) / 2;
      if ( e < S[mi] ) hi = mi; //[lo, mi)
      else lo = mi + 1; //(mi, hi)
   } //出口时，[lo = hi] = M
   return lo - 1; //至此，[lo]为大于e的最小者，故[lo-1] = m即为不大于e的最大者
}
```

lo	mi	hi
0	3	7

版本A

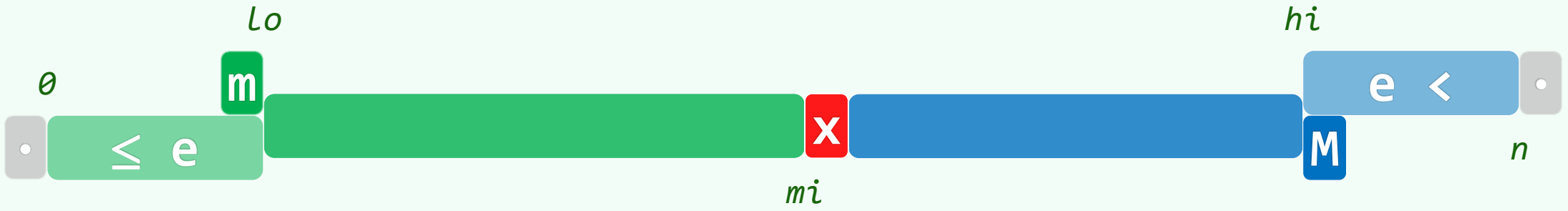
lo	mi	hi
0	3	7
3	5	
5	6	6

版本B

lo	mi	hi
0	3	7
4	5	
6	6	6

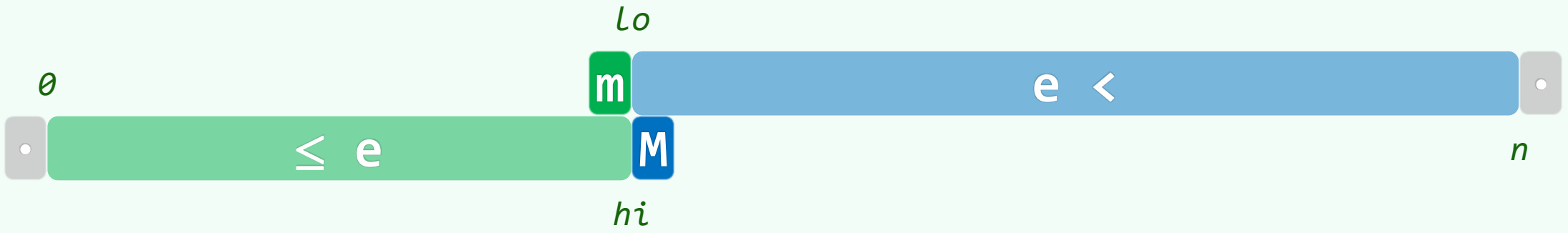
版本C

Loop Invariant: [0,lo) ≤ e < [hi,n)



❖ 只要该不变性始终保持，当算法终止（lo == hi）时

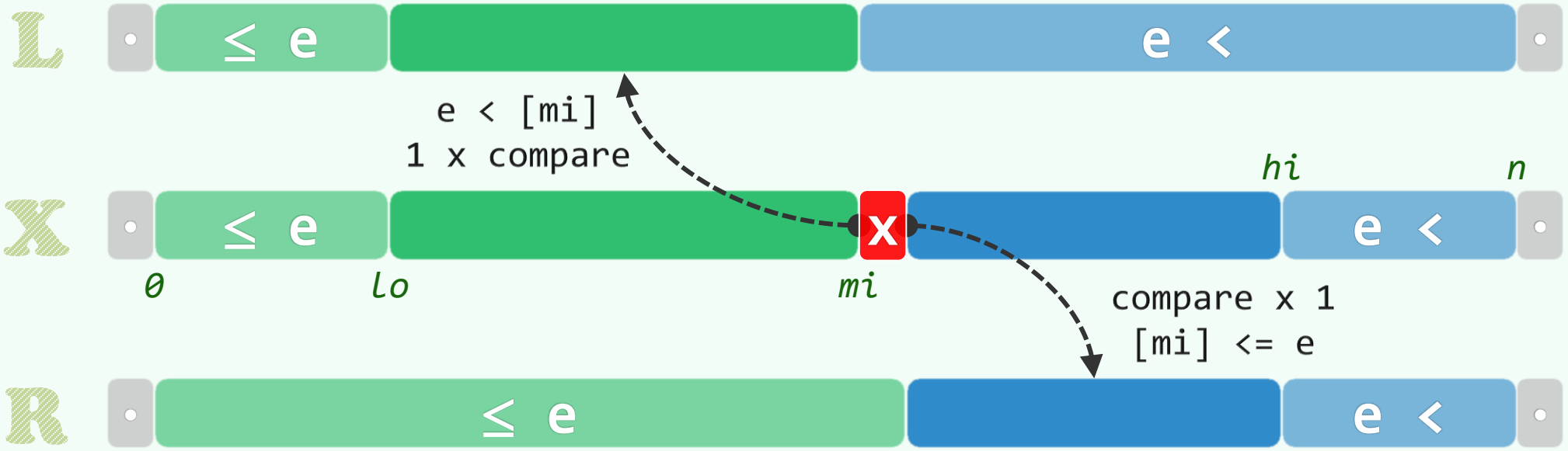
- [lo-1]即是不大于e的最大者：$m = \max(\{-1\} \cup \{\, k \mid S[k] \le e \,\})$
- [hi]即是大于e的最小者：$M = \min(\{\, k \mid e < S[k] \,\} \cup \{n\})$



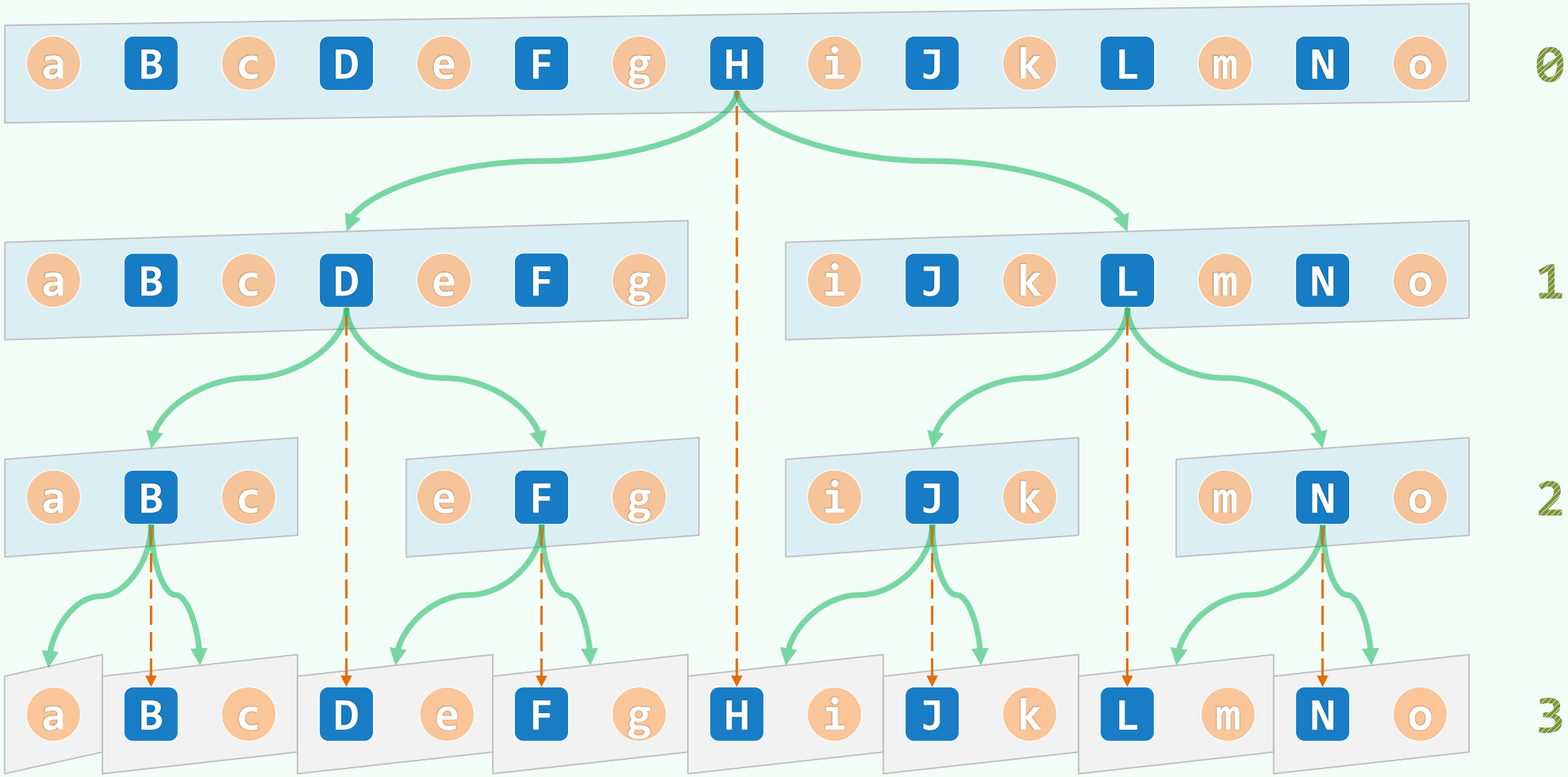
Loop Invariant: [0,lo) ≤ e < [hi,n)

❖ 初始时，lo = 0且hi = n，[0,lo) = [hi,n) = ∅，自然成立

❖ 数学归纳：假设不变性一直保持至(a)；则以下无非两种情况...



平均查找长度：$ASL_{succ} = ASL_{fail} = \mathcal{O}(1.00 \cdot \log n)$



向量

有序向量：插值查找

邓俊辉
deng@tsinghua.edu.cn

原理与算法

❖ 大数定律：越长的序列，元素的分布越有规律

❖ 最为常见：独立同分布且服从均匀分布

[lo,hi]内各元素的秩与其数值会近似地呈线性关系

$$\frac{mi-lo}{hi-lo} \approx \frac{[mi]-[lo]}{[hi]-[lo]}$$

❖ 因此：通过猜测轴点mi，可以极大地提高收敛速度

$$mi \approx lo + (hi-lo)\cdot\frac{e-[lo]}{[hi]-[lo]}$$

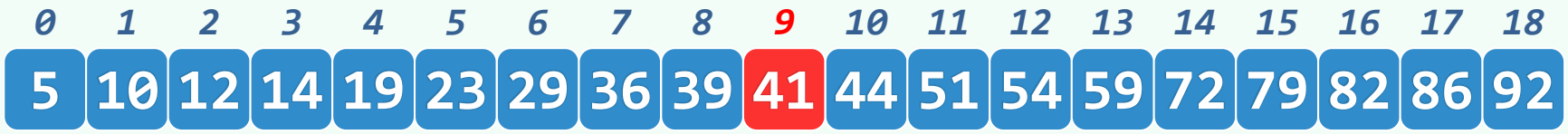
❖ 以英文词典为例：binary大致位于2/26处

search大致位于19/26处

[lo]	0	A	1	[1,53)
	1	B	74	[53,104)
	2	C	158	[104,156)
	3	D	292	[156,208)
	4	E	368	[208,259)
	5	F	409	[259,311)
	6	G	473	[311,363)
	7	H	516	[363,414)
	8	I	562	[414,466)
	9	J	607	[466,518)
	10	K	617	[518,569)
	11	L	628	[569,621)
	12	M	681	[621,673)
	13	N	748	[673,724)
	14	O	771	[724,776)
	15	P	806	[776,827)
	16	Q	915	[827,879)
	17	R	922	[879,931)
	18	S	1002	[931,982)
	19	T	1176	[982,1034)
	20	U	1253	[1034,1086)
	21	V	1271	[1086,1137)
	22	W	1289	[1137,1189)
	23	X	1337	[1189,1241)
	24	Y	1338	[1241,1292)
	25	Z	1341	[1292,1344)
[hi]	26		1344	

实例：查找目标 e = 50

❖ lo = 0, hi = 18



插值：mi = 0 + (18-0)*(50-5)/(92-5) = 9

比较：A[9] = 41 < e

❖ lo = 10, hi = 18



插值：mi = 10 + (18-10)*(50-44)/(92-44) = 11

比较：A[11] = 51 > e

❖ lo = hi = 10



插值：mi = 10

比较：A[10] = 44 < e，返回：10

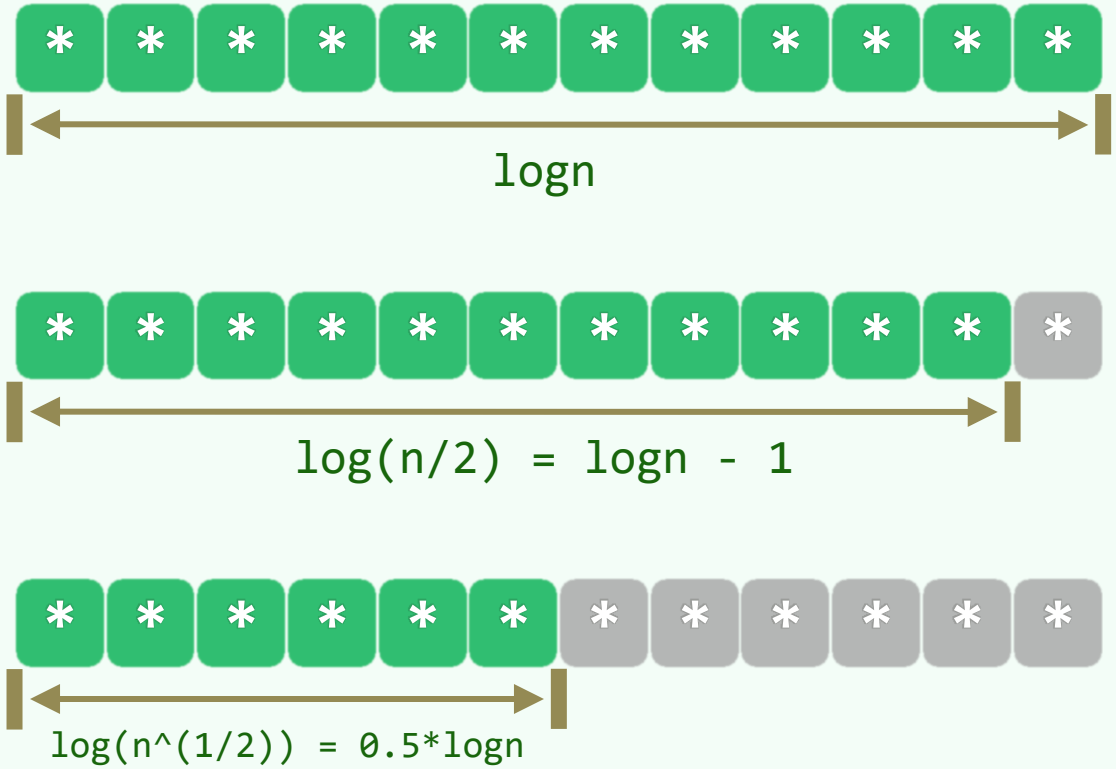
性能

❖ **最坏：** $hi - lo = \mathcal{O}(n)$ //具体实例?

❖ **平均：每经一次比较，待查找区间宽度由 n 缩至 $\sqrt{n}$** //[Yao76, PIA78]

$$n \to \sqrt{n} \to \sqrt{\sqrt{n}} \to \sqrt{\sqrt{\sqrt{n}}} \to \cdots \to 2$$

$$\underbrace{n \to n^{1/2^1} \to n^{1/2^2} \to n^{1/2^3} \to \cdots \to 2}_{\mathcal{O}(\log \log n)}$$



❖ **每经一次比较，**

查找区间宽度的数值n开方，有效字长logn减半

- **插值查找 = 在字长意义上的折半查找**
- **二分查找 = 在字长意义上的顺序查找**

综合评价

❖ 从$\mathcal{O}(\log n)$到 $\mathcal{O}(\log\log n)$优势并不明显

（除非查找表极长，或比较操作成本极高）

比如，n = 2^(2^5) = 2^32 = 4G 时

- $\log_2(n) = 32$
- $\log_2(\log_2(n)) = 5$

❖ 须引入乘法、除法运算

❖ 易受畸形分布的干扰和“蒙骗”

❖ 实际可行的方法是

算法接力

- 首先通过插值查找

  迅速将查找范围缩小到一定的尺度
- 然后再改为二分查找

  进一步缩小范围
- 最后（当数据项只有200～300时）

  改用顺序查找

2．向 量

02-A.抽象数据结构

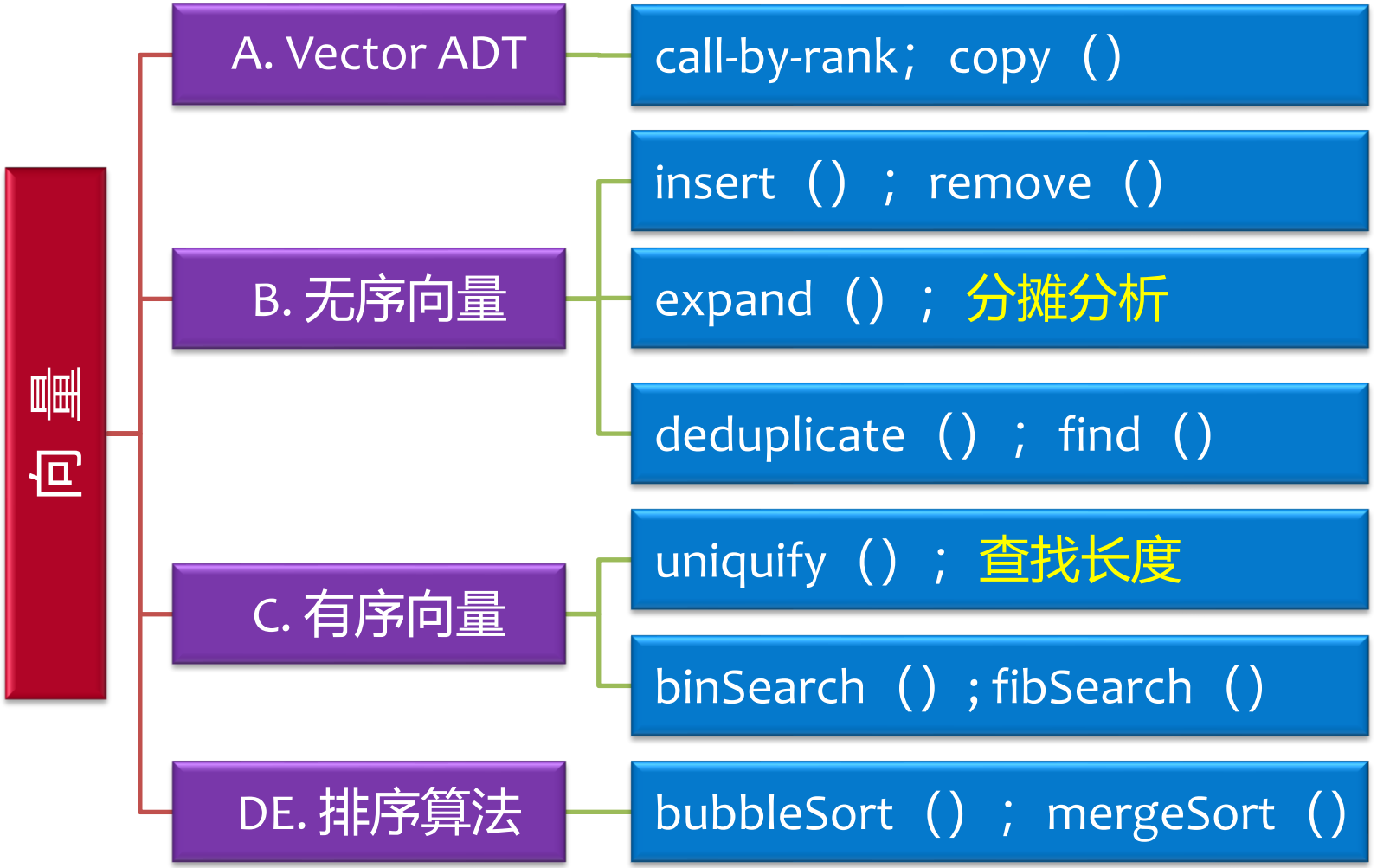
02-B.无序向量

02-C.有序向量

02-D.起泡排序

02-E.归并排序

02-F. 位图



02.Vector

D-起泡排序

向量

起泡排序

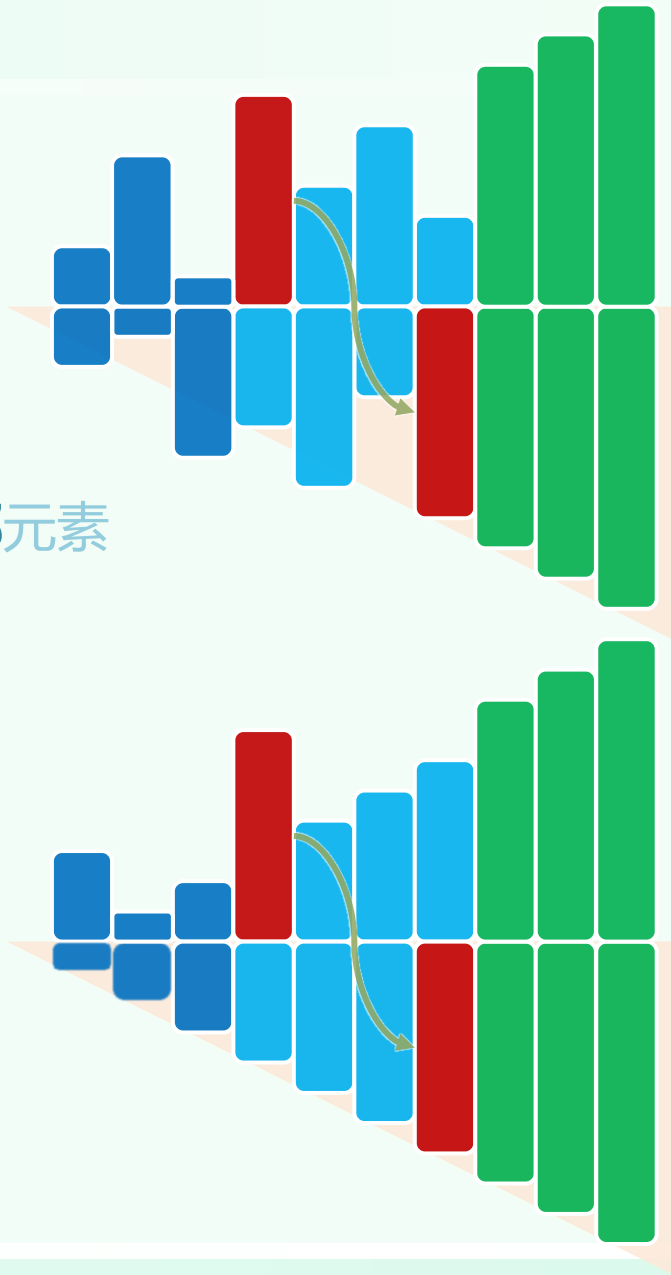
邓俊辉
deng@tsinghua.edu.cn

排序器：统一入口

```
template <typename T> void Vector<T>::sort( Rank lo, Rank hi ) {
   if ( lo + 2 > hi ) return; //退化情况，统一处理
   switch ( rand() % 6 ) {
      case 1 :    bubbleSort( lo, hi ); break; //起泡排序
      case 2 : selectionSort( lo, hi ); break; //选择排序（习题）
      case 3 :     mergeSort( lo, hi ); break; //归并排序
      case 4 :      heapSort( lo, hi ); break; //堆排序（第12章）
      case 5 :     quickSort( lo, hi ); break; //快速排序（第14章）
      default :    shellSort( lo, hi ); break; //希尔排序（第14章）
   } //随机选择算法，以尽可能充分地测试；应用时可视具体问题的特点，灵活确定或扩充
} //sort
```

基本版：用上模板，封装一下

```
template <typename T>
void Vector<T>::bubbleSort( Rank lo, Rank hi ) {
   while ( lo < --hi ) //逐趟起泡扫描
      for ( Rank i = lo; i < hi; i++ ) //逐对检查相邻元素
         if ( _elem[i] > _elem[i+1] ) //若逆序
            swap( _elem[i], _elem[i+1] ); //则交换
}
```

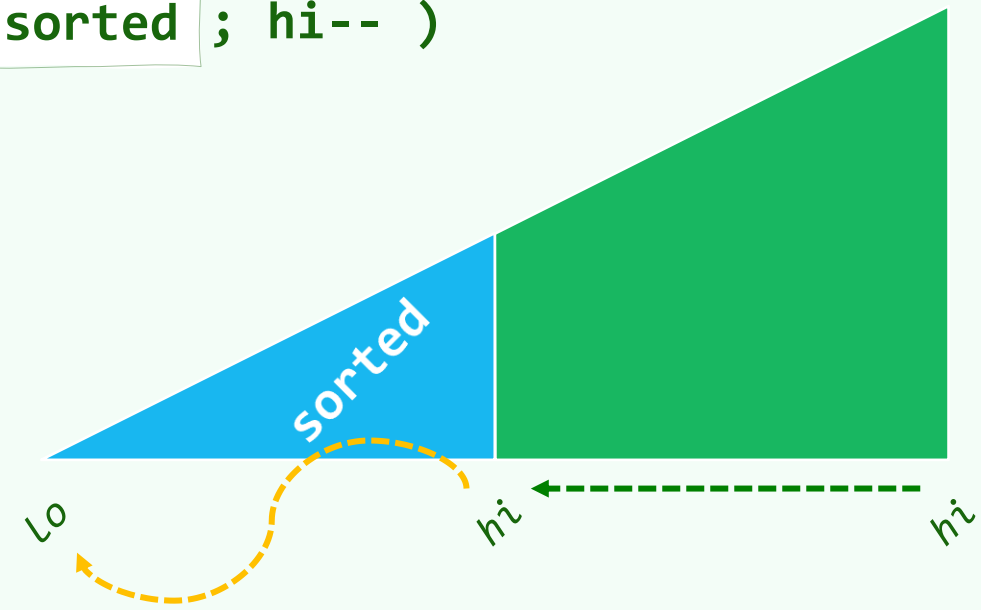


- $\Theta(n^2)$：不识好坏，千篇一律
- 然而，不同输入的混乱程度毕竟有别，比如...

提前终止版

- 有可能...行至中途，前缀已然提前有序|sorted——此时，可以提前收工！
- 关键：如何及时发现这类情况，同时渐近复杂度总体不增呢？
-

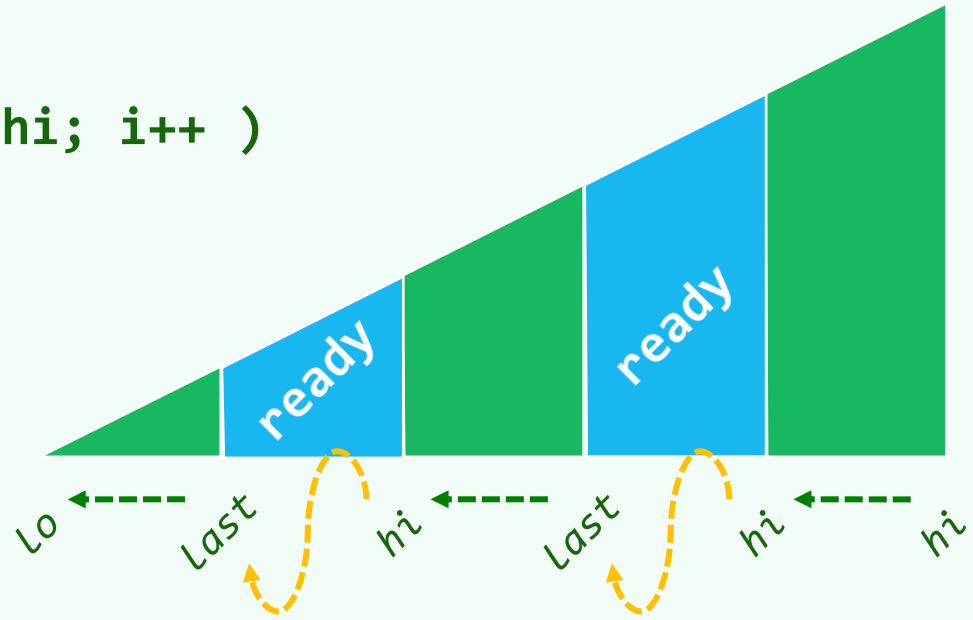
```
template <typename T> void Vector<T>::bubbleSort( Rank lo, Rank hi ) {
    for ( bool sorted = false; sorted = !sorted; hi-- )
        for ( Rank i = lo+1; i < hi; i++ )
            if ( _elem[i-1] > _elem[i] ) {
                swap( _elem[i-1], _elem[i] );
                sorted = false;
            }
} //有改进，但仍有继续改进的余地，比如...
```



跳跃版

- 在前一趟扫描过程中，若最后被交换的是[last-1]和[last]，则[last,hi)必然均已就位
- 此时，可以跳过这个后缀，直接对[lo,last)继续做起泡扫描
-

```
template <typename T> void Vector<T>::bubbleSort( Rank lo, Rank hi ) {
    for ( Rank last ; lo < hi; hi = last )
        for ( Rank i = (last = lo) + 1; i < hi; i++ )
            if ( _elem[i-1] > _elem[i] ) {
                swap( _elem[i-1], _elem[i] );
                last = i;
            }
} //断言: [lo,last) < [last] <= (last,hi)
```



综合评价

❖ 时间效率：最好$O(n)$，最坏$O(n^2)$

❖ 排序算法的**稳定性**|stability，是更为细致的要求

相等的元素在输入、输出序列中的相对**次序**，是否保持**不变**?

- 输入： 6, 7a, 3, 2, 7b, 1, 5, 8, 7c, 4
- 输出： 1, 2, 3, 4, 5, 6, 7a, 7b, 7c, 8 //stable

  1, 2, 3, 4, 5, 6, 7a, 7c, 7b, 8 //unstable

❖ 以上起泡排序算法是**稳定**的吗？是的！

- 排序过程中，唯有**相邻**元素才可交换
- 比较相邻元素时，是用严格不等号">"，而非">="

02.Vector

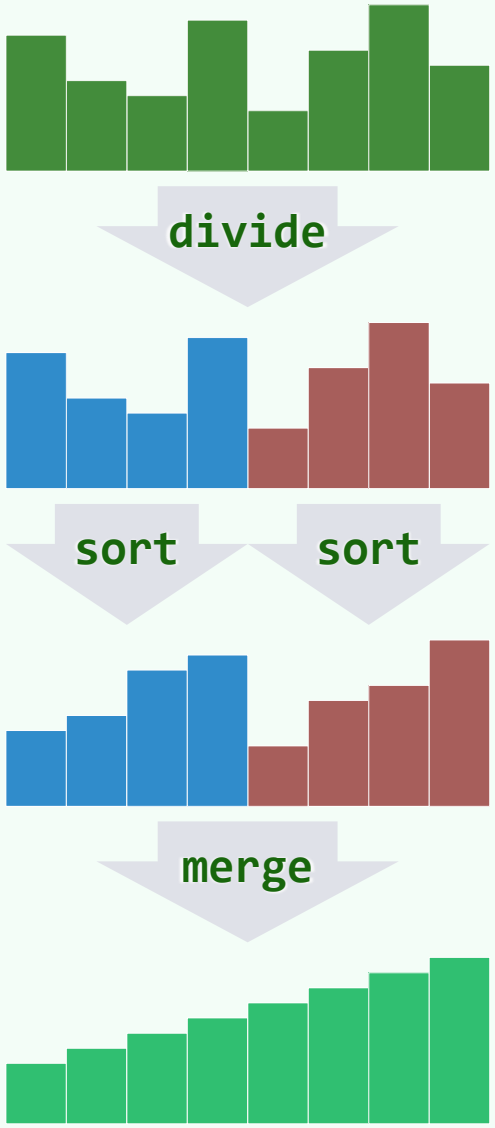
E-归并排序

向量

归并排序

邓俊辉
deng@tsinghua.edu.cn

分而治之：原理



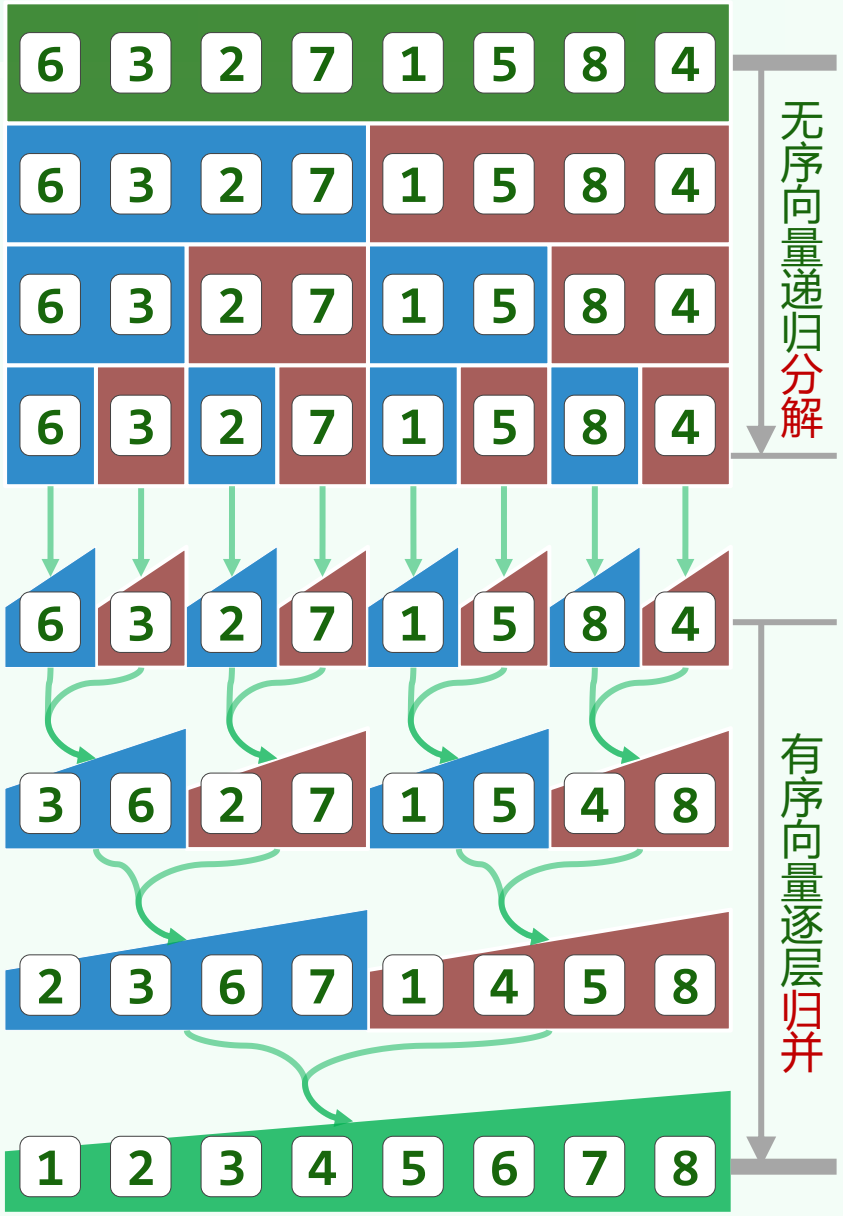
- 向量与列表通用
- J. von Neumann

  1945年首次编程实现

  - 序列均分为二 //O(1)
  - 子序列递归排序 //2×T(n/2)
  - 合并有序子序列 //O(n)
- 前两步毫无技巧可言

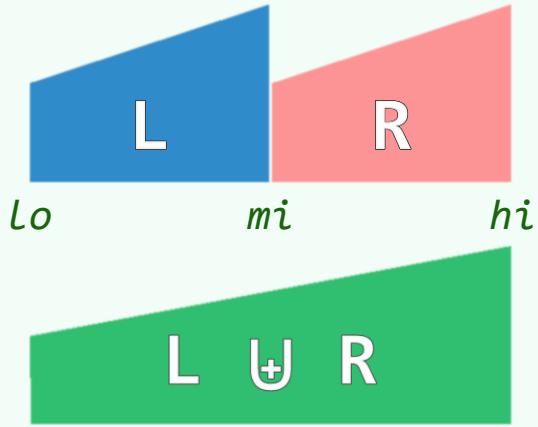
  最后一步才是实质的





分而治之：主算法

```
❖ template <typename T> void Vector<T>::mergeSort( Rank lo, Rank hi ) {
     if ( hi - lo < 2 ) return; //单元素自然有序，否则
     Rank mi = (lo + hi) / 2; //均分为前缀、后缀
     mergeSort( lo, mi ); mergeSort( mi, hi ); //各自排序
     merge( lo, mi, hi ); //归并（实质的计算），O(n)
  } //O(nlogn)
```

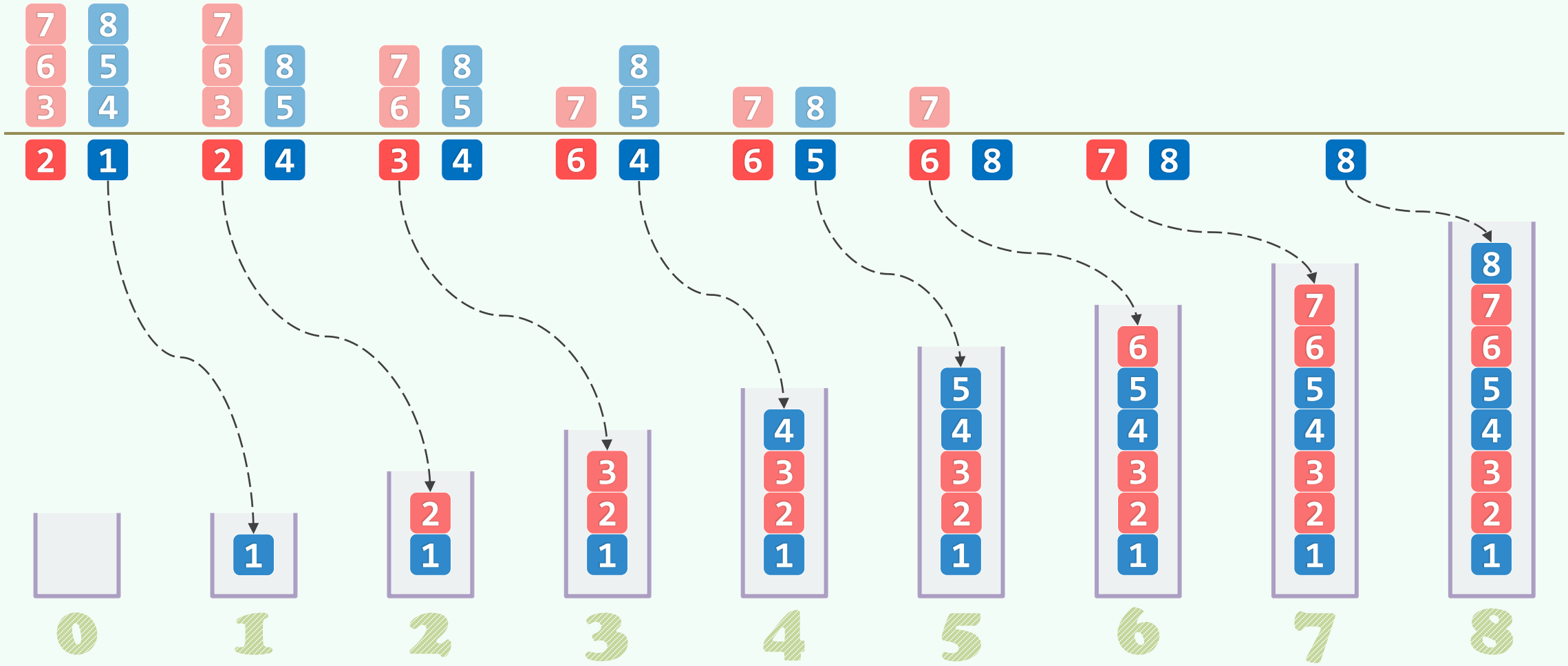


❖ 我们马上就会看到，归并能在线性时间内完成 //不仅向量，列表也可以！

那么整体复杂度即为 $\mathcal{O}(n \cdot \log n)$ ！ //worst-case optimal

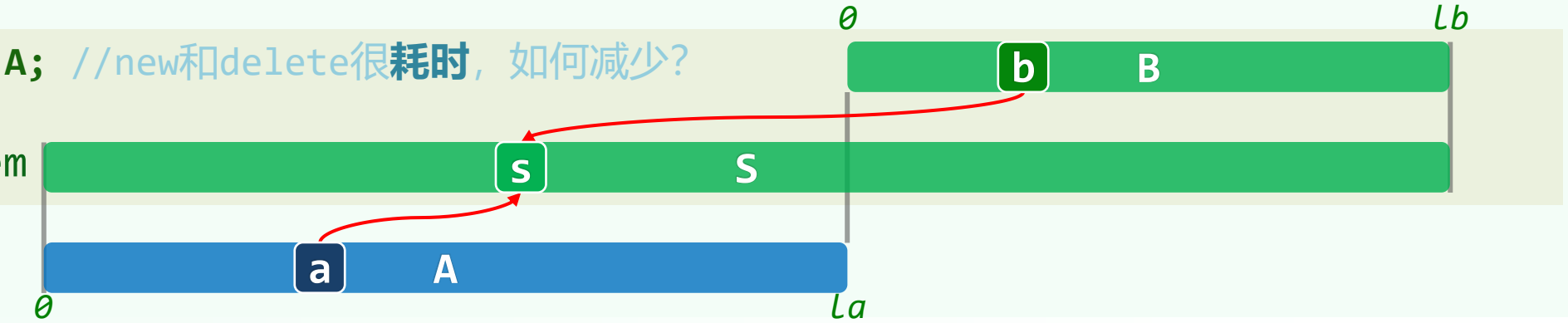
❖ 依然是...减治：首先，关注全局最小的那个元素...

二路归并：减而治之

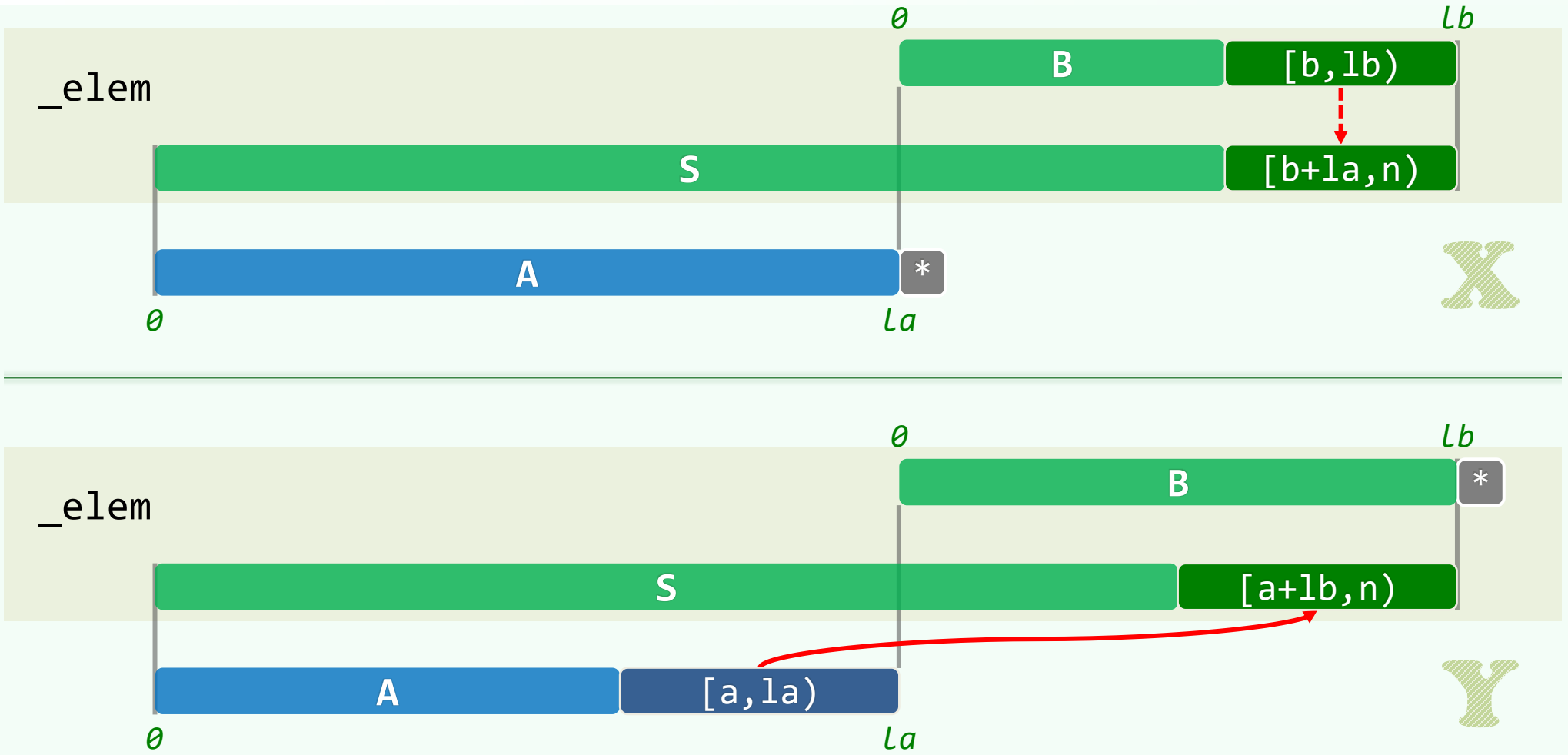


二路归并：实现

```
template <typename T> void Vector<T>::merge(Rank lo, Rank mi, Rank hi) {
   Rank s = 0; T* S = _elem+lo; //就地归并至序列S[0,hi-lo)
   Rank a = 0, la = mi-lo; T* A = new T[la]; for (Rank i=0; i<la; i++) A[i] = S[i]; //复制前缀
   Rank b = 0, lb = hi-mi; T* B = S+la; //直接使用后缀
   while ((a < la) && (b < lb)) //反复地比较A、B的首元素
      S[s++] = (A[a] <= B[b]) ? A[a++] : B[b++]; //小者优先归入S
   while (a < la) S[s++] = A[a++]; //若B先耗尽，则将A残余的后缀归入S中——若A先耗尽呢?
   delete[] A; //new和delete很耗时，如何减少?
}
```



二路归并：正确性 + 复杂度



❖ 无论过程如何，最终a+b = la + lb = n，运行时间总是 $\mathcal{O}(n)$；由主定理，归并排序为 $\mathcal{O}(n \log n)$

优点 + 不足

- 最坏情况最优$\mathcal{O}(n \log n)$的首个排序算法
- 不需随机读写，完全顺序访问——尤其

  适用于列表之类的序列、磁带之类的设备
- 只要实现恰当，可保证稳定
- 可扩展性极佳，十分适宜于外部排序
- 同层子任务彼此独立，易于并行化
- 反复new/delete，耗时

  ——可优化为只做一次
- 需要$\Omega(n)$辅助空间

  ——适当多花时间，可令就地
- 即便已接近甚至完全有序，仍需$\Omega(n \log n)$时间

  ——可优化至$\mathcal{O}(n)$，同时其它情况不受影响
- 序列足够短之后，递归效率骤降

  ——适时变招，改用bubbleSort()

2．向 量

02-A.抽象数据结构

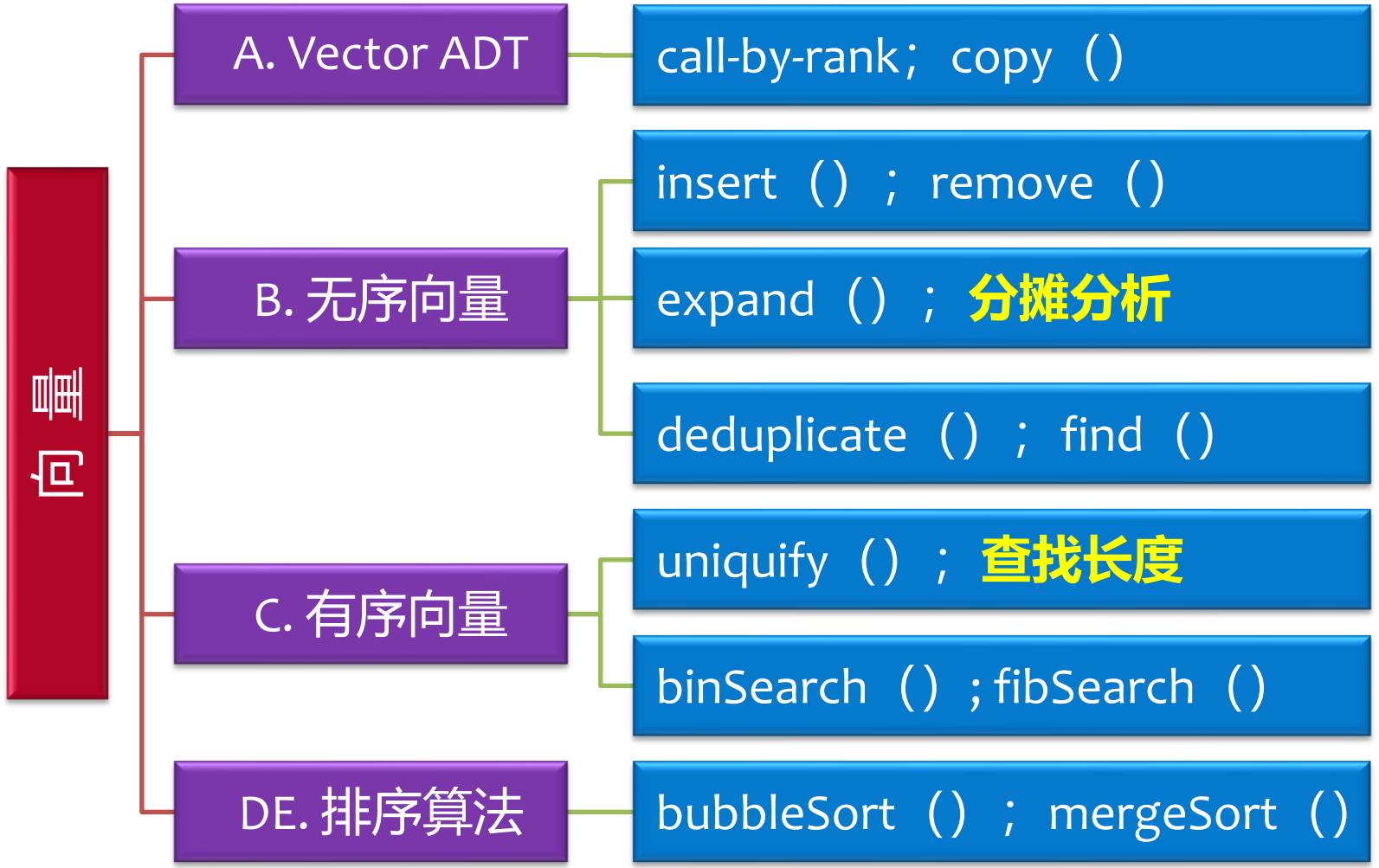
02-B.无序向量

02-C.有序向量

02-D.起泡排序

02-E.归并排序

02-F. 位图



BitMap位图简介

- 位图（Bitmap）是一种**紧凑型数据结构**，用于表示一个固定大小的集合或序列中的元素状态（存在或不存在）。它通常用于处理一组整数值或布尔值，例如集合操作、数据筛选和计数等应用场景。
- 位图通过使用**位数组**来表示集合中的元素状态，数组中的每一位（bit）表示集合中的一个元素存在与否，从而实现高效的时间效率和空间性能。
- 位图的大小通常与要表示集合的元素范围成正比，它可以快速执行集合操作（如并集、交集和差集）以及检查元素是否存在于集合中。
  - 位图使用位运算（如与、或、异或）来高效地执行集合操作

BitMap位图简介

❖位图的优点

- 高效的利用空间：位图在表示大范围的数据时非常紧凑，每个元素只需要一个位。
- 快速的集合操作：位图支持高效的并集、交集和差集操作，这些操作可以通过位运算来实现。
- 快速的存在检查：位图可以快速检查某个元素是否存在于集合中，通过索引直接访问位数组。

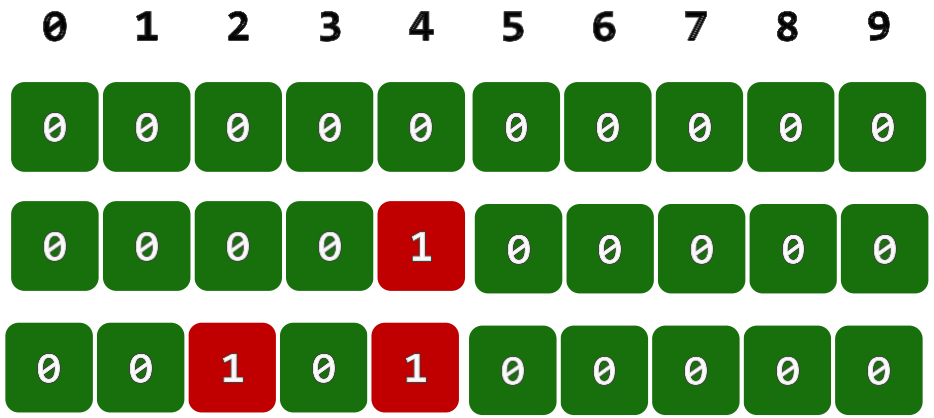
❖位图的缺点

- 固定大小：位图通常适用于固定大小的集合。如要集合范围不固定，可能需要额外的空间。
- 不适合稀疏数据：位图在表示稀疏数据时可能浪费大量空间，因为空闲元素仍然会占用空间。

BitMap位图简介

❖BitMap算法举例

- 给定长度是10的bitmap，对应着从0到9的10个整型数，初始bitmap的所有位都是0
- 把整型数4存入bitmap，对应存储的位置就是秩为4的位置，将此bit置为1
- 把整型数2存入bitmap，对应存储的位置就是秩为2的位置，将此bit置为1
- 此时，Bitmap不仅方便查询存储了哪些元素，还可以方便去重



02.Vector

F-位图

向量

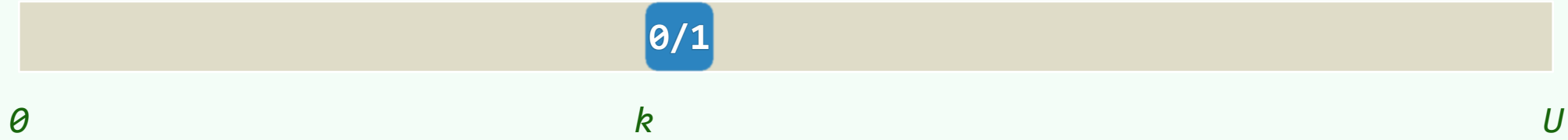
位图：结构

邓俊辉
deng@tsinghua.edu.cn

有限整数集：秩 ~ 整数

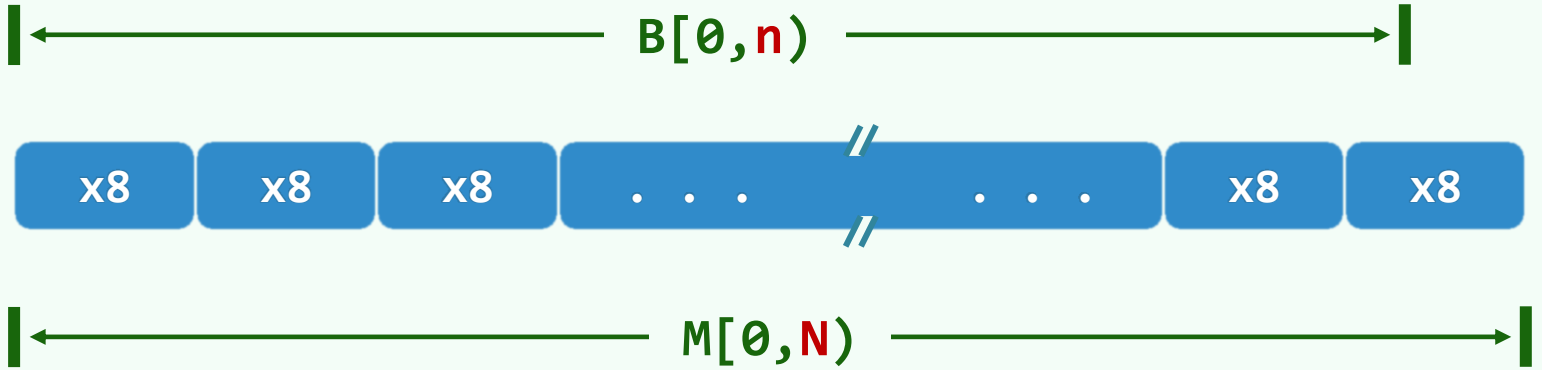
$\forall\, 0 \le k < U$:

$k \in S$?	`bool test( Rank k );`
$S \cup \{k\}$	`void set( Rank k );`
$S \backslash \{k\}$	`void clear( Rank k );`

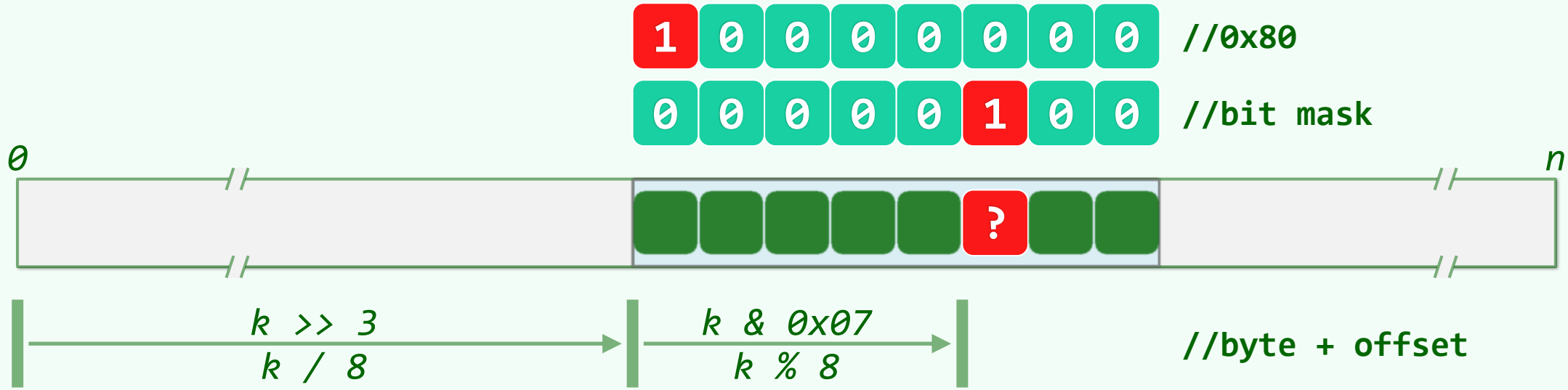


结构

```
struct Bitmap {
   unsigned char* M;
   Rank N, size;
   init( Rank n ) { M = new uint8_t[ N = (n+7)/8 ]; memset( M, 0, N ); }
   Bitmap( Rank n = 64 ) { init( max(64U, n) ); _sz = 0; }
   ~Bitmap() { delete[] M; }
   bool test( Rank k );  void set( Rank k );  void clear( Rank k );
   void expand( Rank k );
};
```



实现




```
bool test (Rank k) {    expand(k); return  M[k>>3]  &  (0x80 >> (k&0x07));          }

void set  (Rank k) { if ( test(k)) return; M[k>>3] |=  (0x80 >> (k&0x07)); _sz++; }

void clear(Rank k) { if (!test(k)) return; M[k>>3] &= ~(0x80 >> (k&0x07)); _sz--; }
```

向量

位图：典型应用

邓俊辉
deng@tsinghua.edu.cn

去重：大数据+小集合

❖ 老问题：int A[n]的元素均取自[0,m)

如何剔除其中的相等者？

❖ 仿照Vector::uniquify()

先排序，再扫描

—— $\mathcal{O}(n \log n + n)$ —— 毫无压力

❖ 新特点：数据体量虽大，却有大量元素相等

- 想想我们电脑里的jpg、mp3、mp4...
- 还有，朋友圈...

❖ 比如，10,000,000,000个24位无符号整数

$$2^{24} = m \ll n = 10^{10}$$

必有大量的相等者

❖ 如果采用内部排序算法

至少需要 4*n = 40GB 内存

- 即便能够申请到这么多空间
- 频繁的I/O也将导致整体效率的低下

❖ 那么，$m \ll n$ 的条件，又应如何加以利用？

去重：借助位图

```
Bitmap B( m ); //O(m)

for ( Rank i = 0; i < n; i++ )
   B.set( A[i] ); //O(n)

for ( Rank k = 0; k < m; k++ )
   if ( B.test( k ) )
      output(k); //O(m)
```

❖ 总体运行时间 $= \mathcal{O}(n+m) = \mathcal{O}(n)$

❖ 空间 $= \mathcal{O}(m)$

- 就上例而言，降至：

  2^24 = 16*10^6 bit ~ 2MB << 40GB

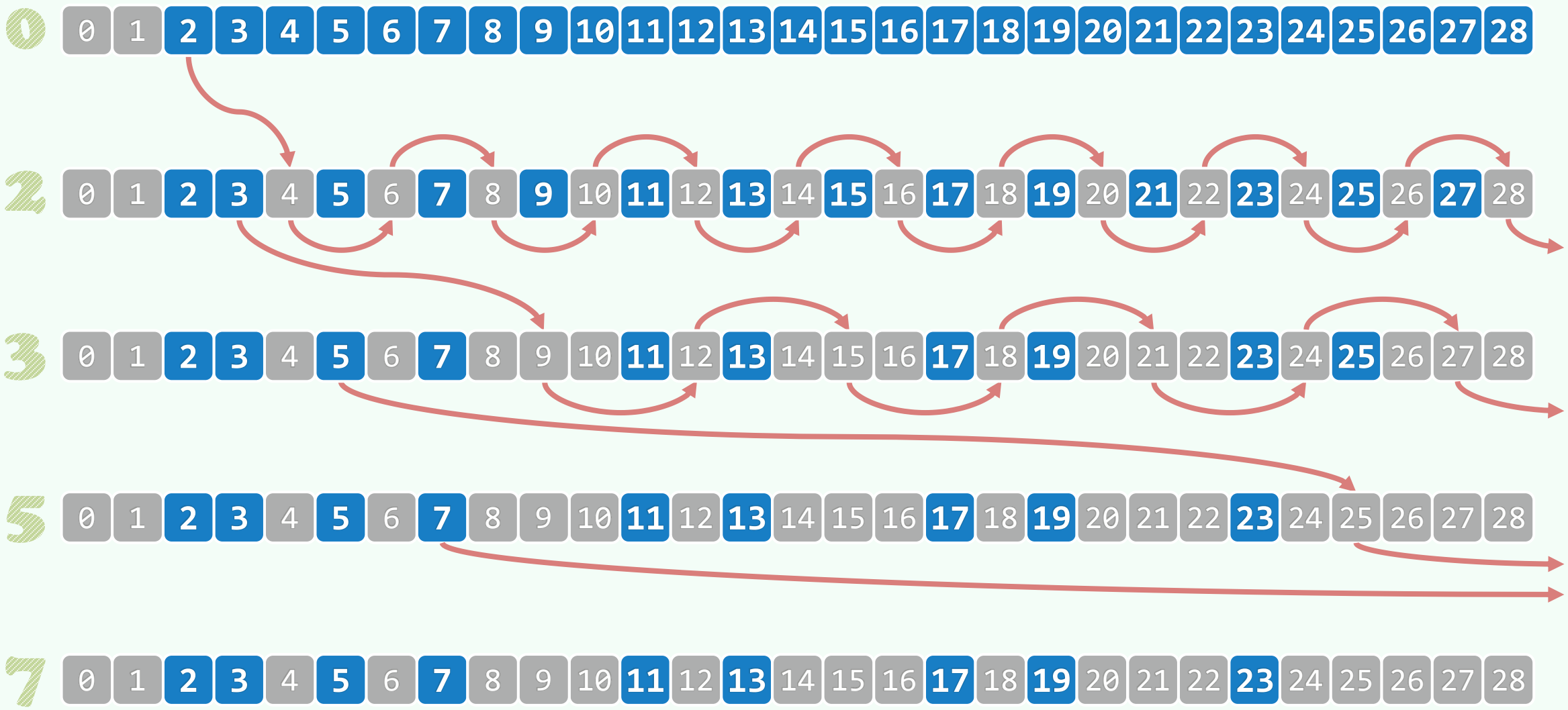
- 即便 m = 2^32，也不过：

  2^29 byte = 0.5GB

❖ 关键在于，如何将查询词表

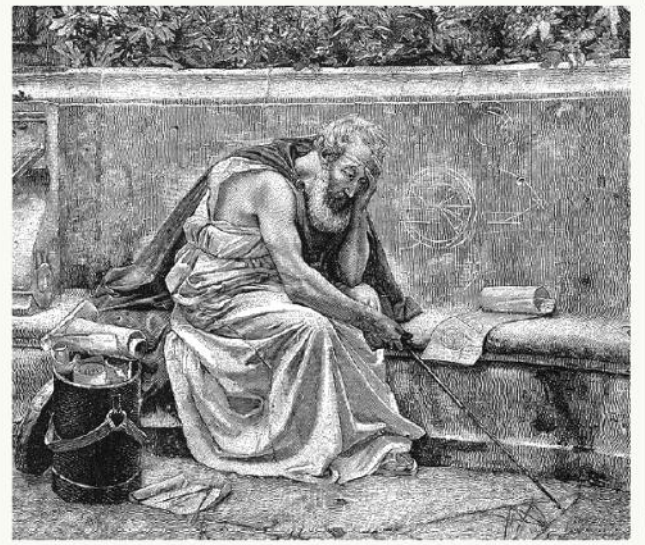
转换为某一整数集合 //保持兴趣

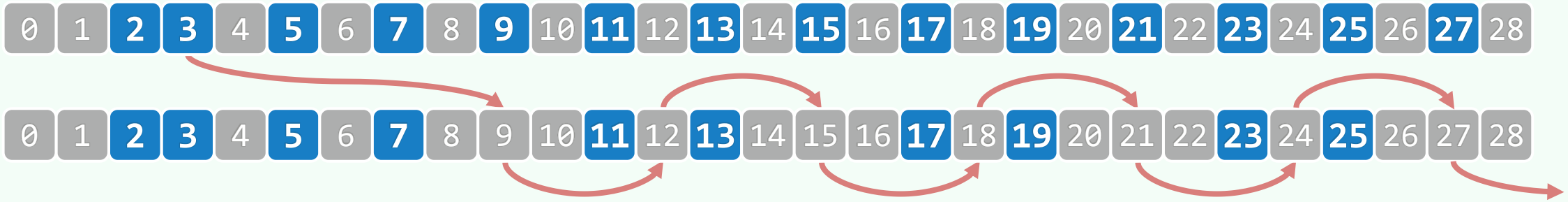
筛法：原理


0
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
2
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
3
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
5
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
7
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28

筛法：实现

```
❖ void Eratosthenes( Rank n ) {
     Bitmap B(n);
     for (Rank r = 2; r < n; r++ ).set(r);
     for ( Rank i = 2; i < n; i++ )
        if ( B.test( i ) )
           for ( Rank j = i*i; j < n; j += i )
              B.clear( j );
  }
```



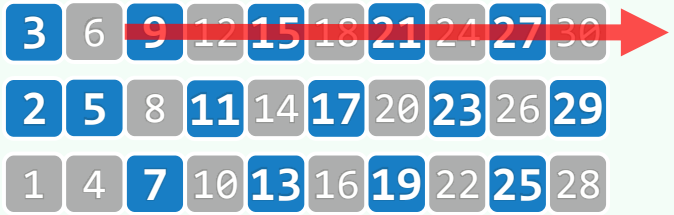


筛法：复杂度

- 由素数定理，内循环启动$n/\ln n$次；每次迭代$\mathcal{O}(n/i)$步
- 记小于n的最大素数为p(n)



$$
\begin{aligned}
& n/2 + n/3 + n/5 + n/7 + n/11 + \ldots + n/p(n) \\
< \; & n/2 + n/3 + n/4 + n/5 + n/6 + \ldots + n/(n/\ln n) \\
= \; & \mathcal{O}(n \cdot (\ln(n/\ln n) - 1)) \\
= \; & \mathcal{O}(n \cdot \ln n - n \cdot \ln(\ln(n))) \\
= \; & \mathcal{O}(n \cdot \log n)
\end{aligned}
$$





向量

位图：快速初始化

邓俊辉
deng@tsinghua.edu.cn

O(n) ~ O(1)

- Bitmap的构造函数中，总要对数据区统一清零

  尽管是通过memset(M,0,N)来完成，但与逐位清零一样，也需要 $\mathcal{O}(N) = \mathcal{O}(n)$ 时间！

- 虽然这并不会影响以上实例的渐近复杂度，但并非所有场合都是如此，比如
  - 第09章的散列表，初始化的复杂度将直接影响到整体效率
  - 第13章的bc[]表构造算法，需要 $\mathcal{O}(|\Sigma| + m) = \mathcal{O}(s + m)$ 时间

    若能省去初始化，则只需 $\mathcal{O}(m)$ 时间
  - 借助Bitmap，固然可以判断1,000,000个32位整数中是否存在相等者

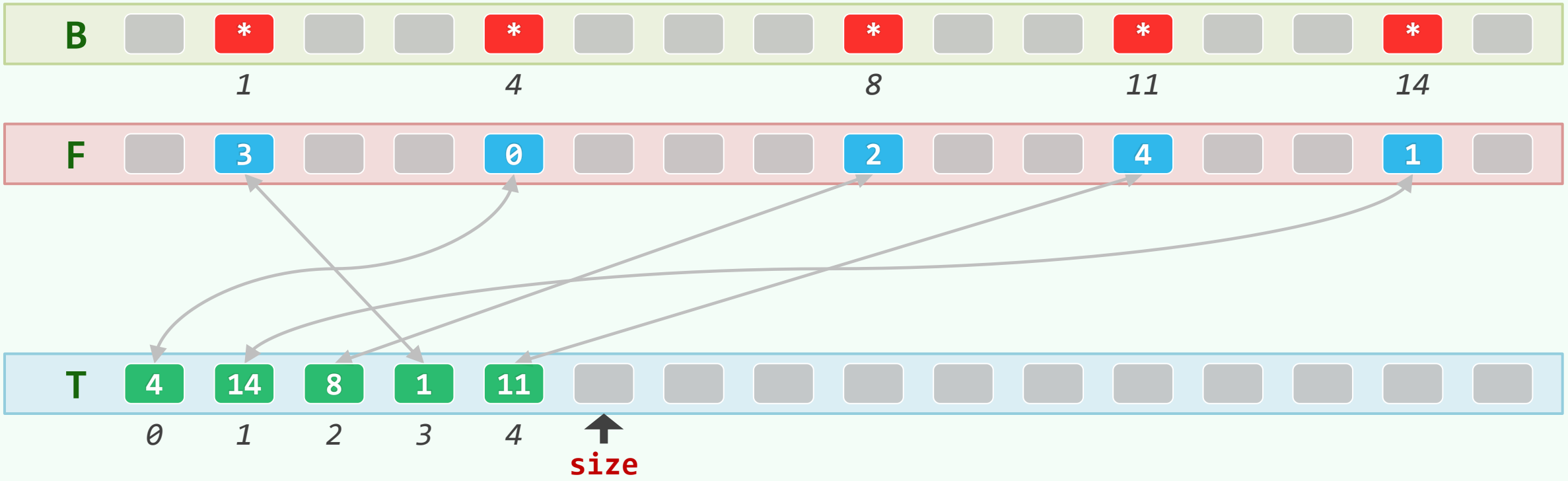
    但2^32 = 4*10^9个bit位清零所需的时间，远远超过接下来对10^6个整数的遍历

- 在这类场合，可否...加速...甚至...免除...数据区的初始化?

J. Hopcroft, 1974: Rank F[N], T[N], size = 0;

❖ 为区分三种状态，需要记录更多信息：**拆分数组 + 校验环**：

```
To[From[k]] == k  &  From[To[i]] == i  //君右杜左，虎符会合
```

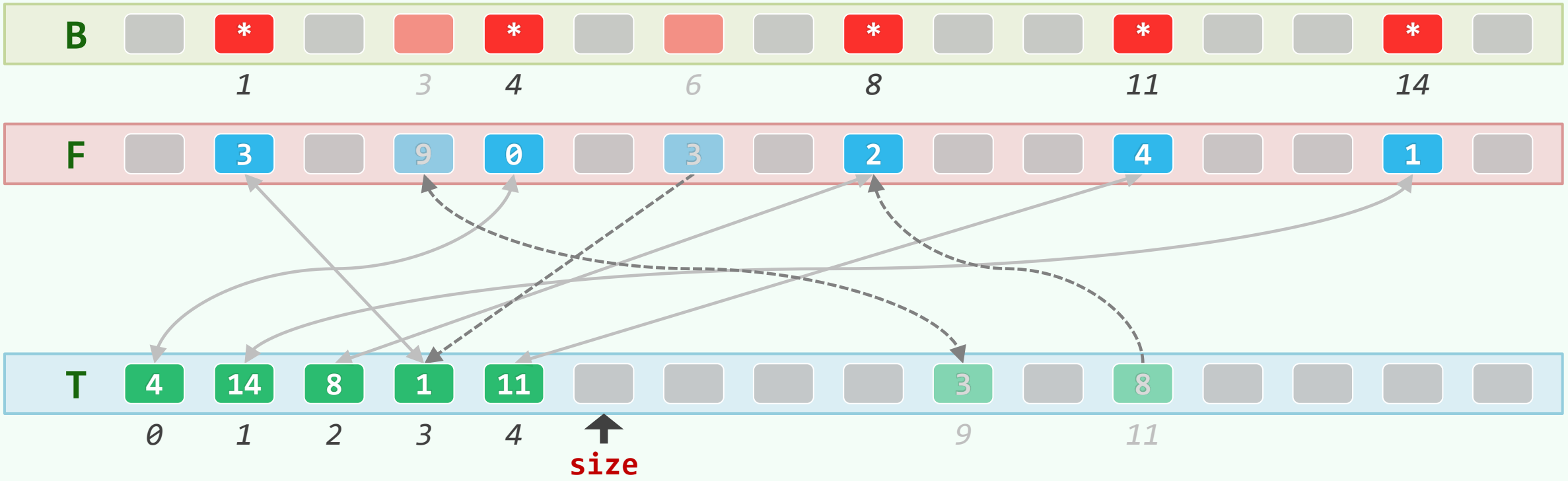


B[m],F[m],T[m]

- ❖内部潜规则：等长向量，rank可定位。
- ❖B[m]：存储0/1，表示某个元素是否存在
- ❖F[m],T[m]中存储的都是rank
- ❖T[m]中存储的是B[m]中标识为1的rank，顺序存储，top是1标识和0标识的分界点
- ❖F[m]中存储的是T[m]中有效的rank，与B[m]通过rank可以相互对应。
  - ➢ 例如，F[1]中存储的是3，即T[3]中存储的就是B中的rank 1.
- ❖即通过rank，F存储了相应的B的秩在T中存放的地址。

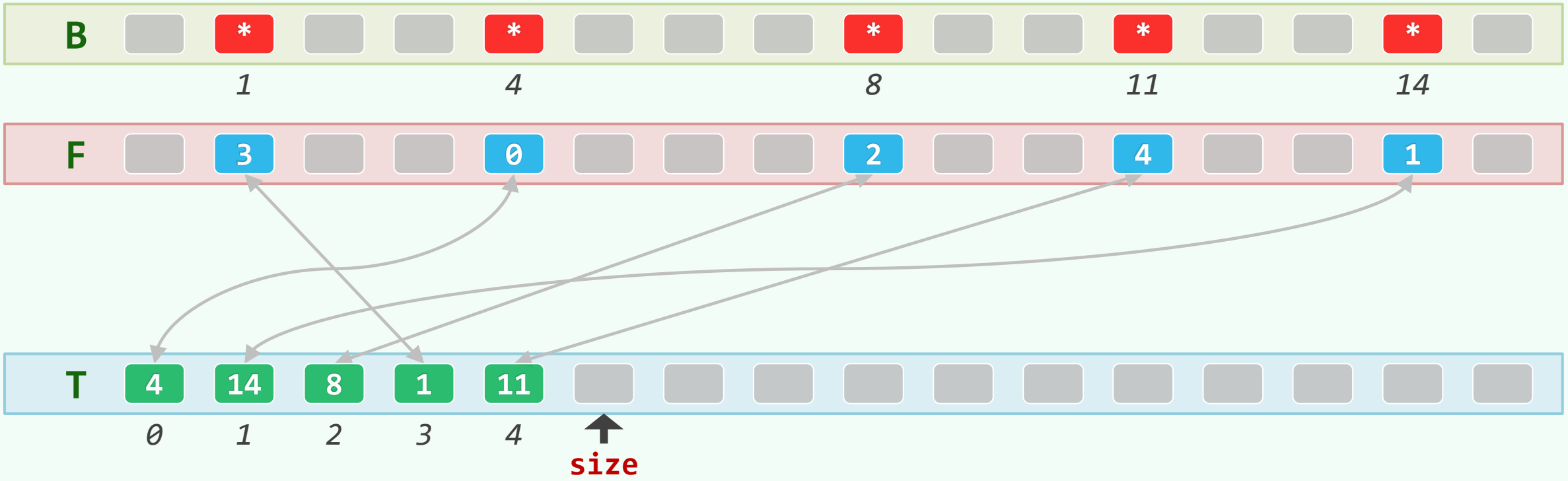
测试: test(1|4|8|11|14) = true & test(3|6) = false

```
bool Bitmap::test( Rank k ) //为剔除随机构成的校验环，还需限制为前缀T[0,size)
   { return (F[k] < size) && (k == T[F[k]]); }  //充要条件
```



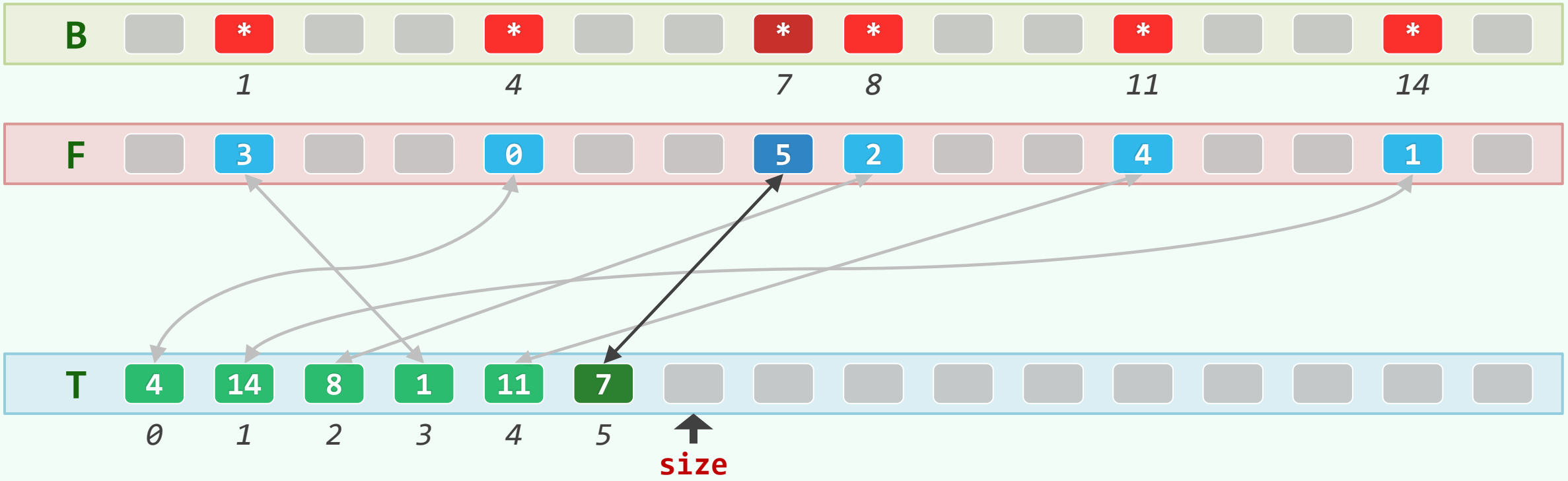
O(1)复位

```
void Bitmap::reset() //前缀清空，即是位图复位
   { size = 0; }
```



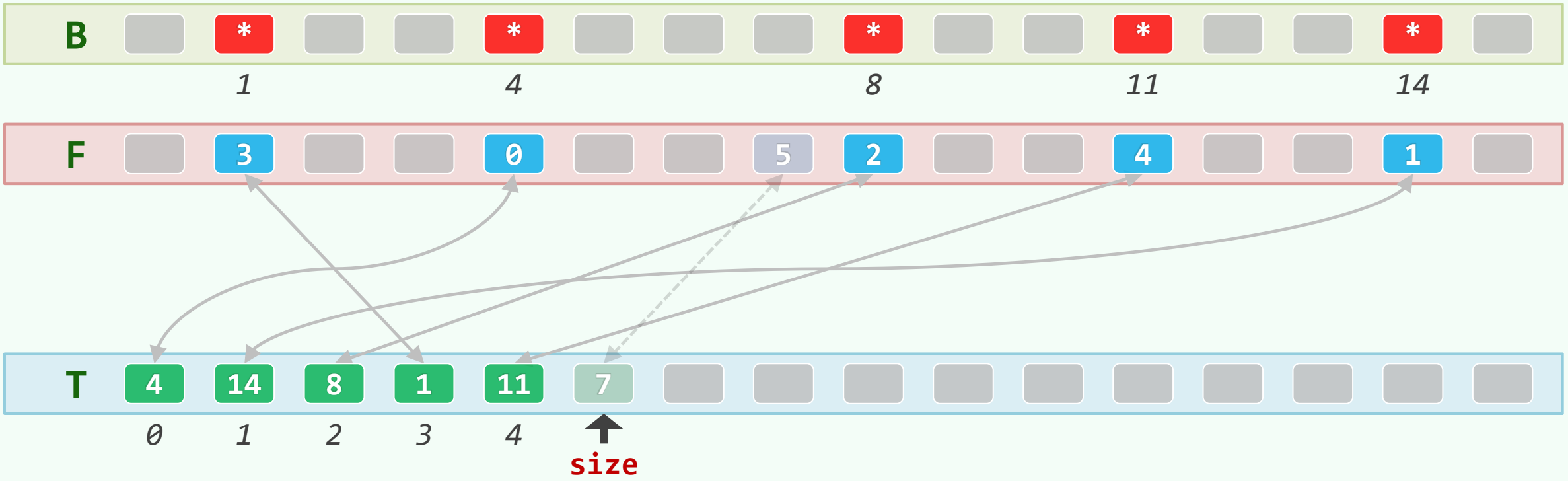
O(1)插入：set(7)

```
void Bitmap::set( Rank k ) //在T尾部，创建一个校验环
   { if ( !test(k) ) { T[size] = k; F[k] = size++; } }
```



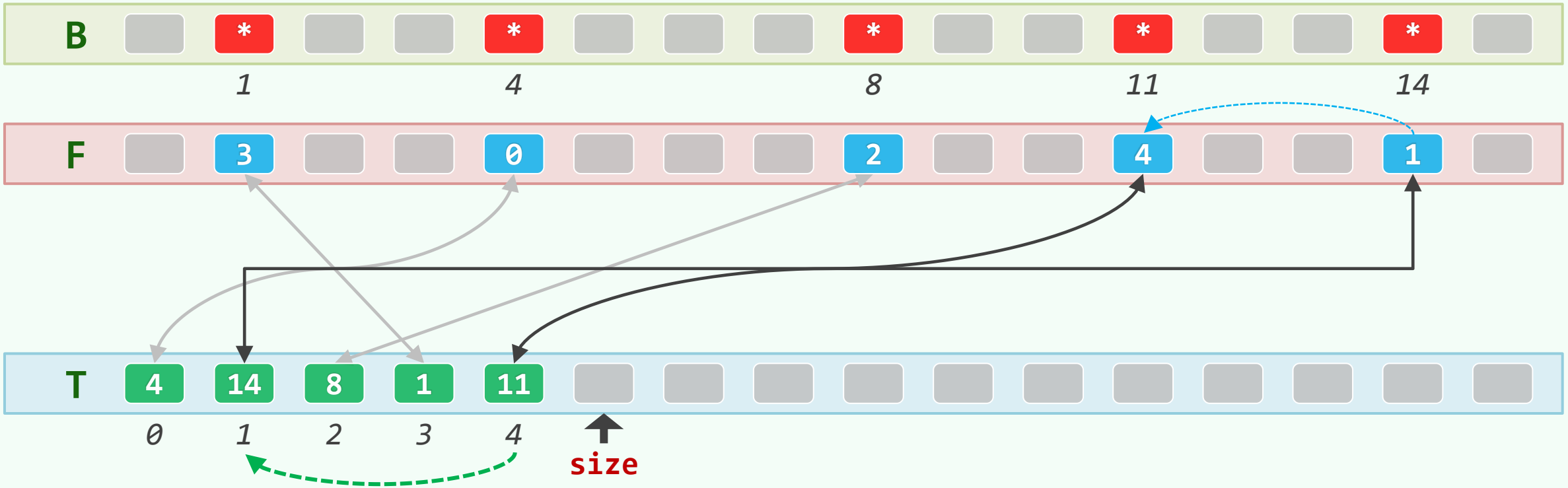
O(1)删除：简单情况，删除最新创建的（在T中拖尾的）校验环：clear(7)

```
void Bitmap::clear( Rank k )
   { if ( test(k) && --size )  { F[T[size]] = F[k]; T[F[k]] = T[size]; } }
```



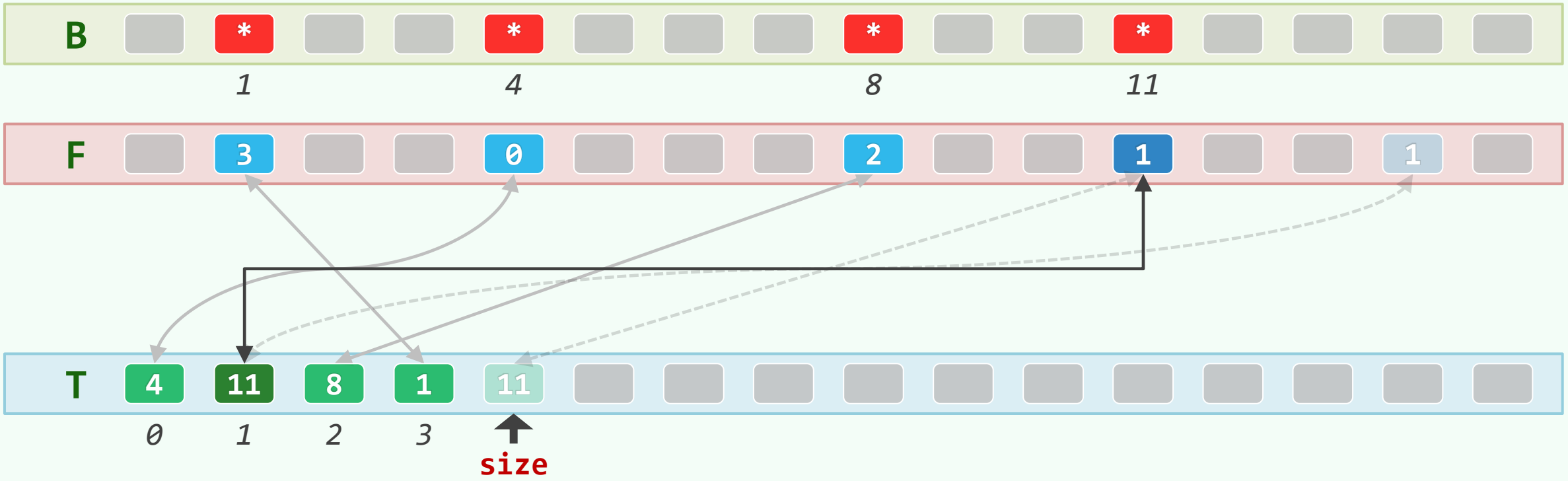
O(1)删除：一般情况，令校验环k，与拖尾的校验环交换：clear(14) ...

```
void Bitmap::clear( Rank k )
   { if ( test(k) && --size )  { F[T[size]] = F[k]; T[F[k]] = T[size]; } }
```



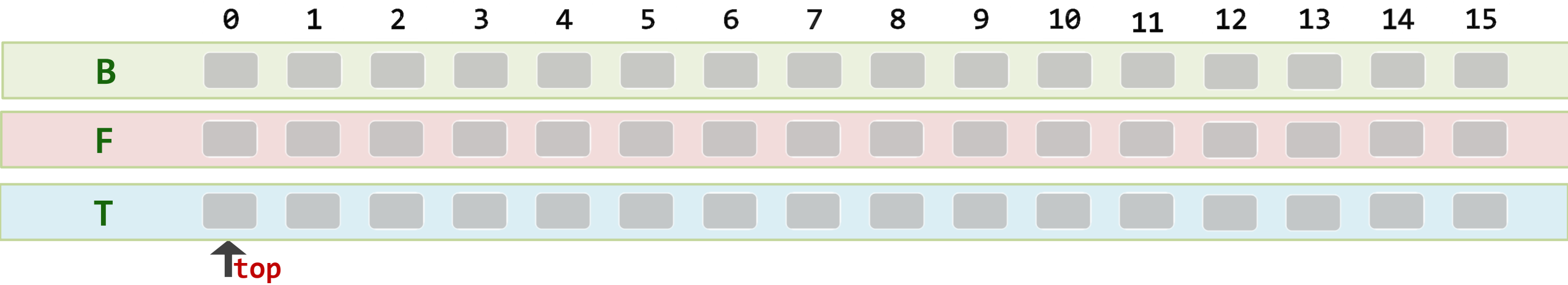
O(1)删除: ... clear(14)

```
void Bitmap::clear( Rank k )
   { if ( test(k) && --size )  { F[T[size]] = F[k]; T[F[k]] = T[size]; } }
```

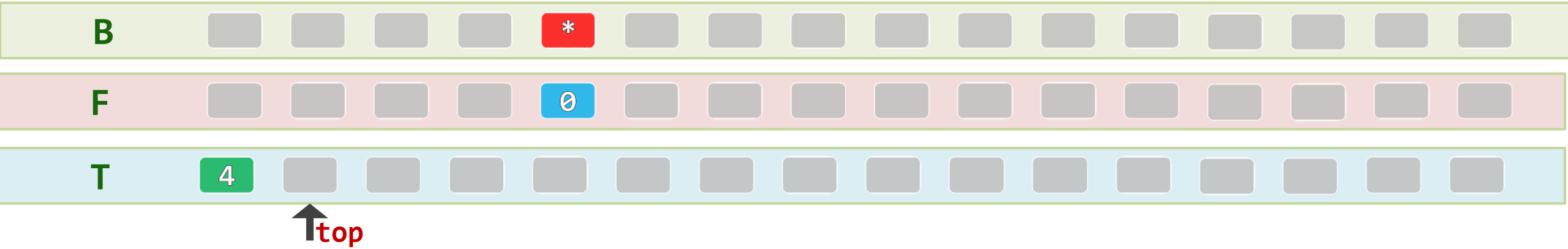


Bitmap举例: reset(); set(4)

❖ `void Bitmap::reset() { top = 0; }` //前缀清空，即是位图复位

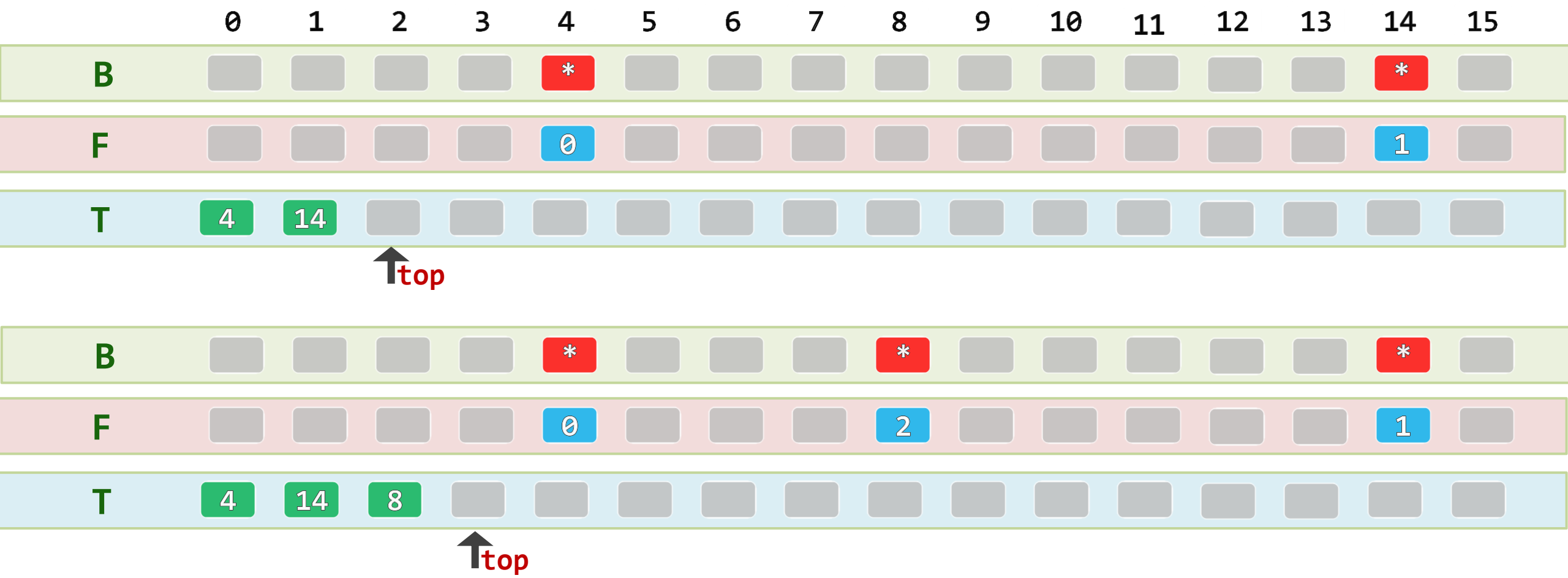


❖ `void Bitmap::set( Rank k ) { if ( !test( k ) ) { T[top] = k; F[k] = top++; } }`



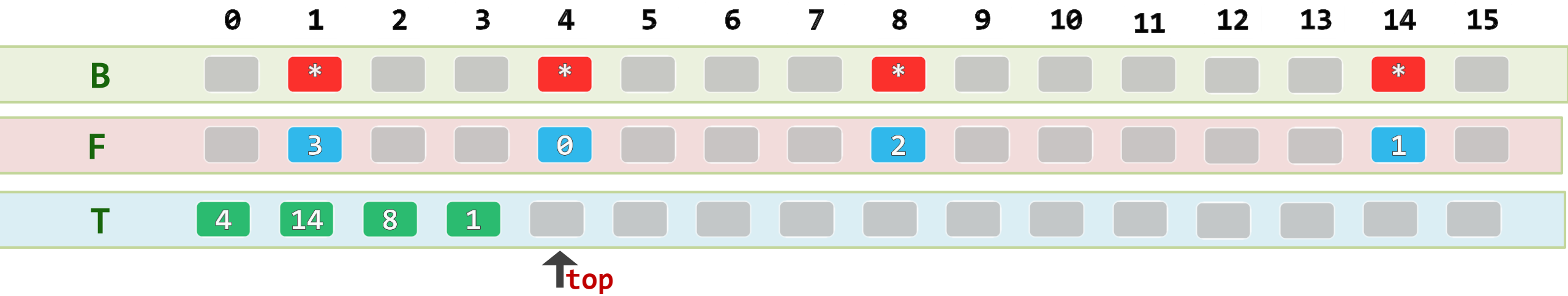
Bitmap举例: set(14); set(8)

❖ `void Bitmap::set( Rank k ) { if ( !test( k ) ) { T[top] = k; F[k] = top++; } }`

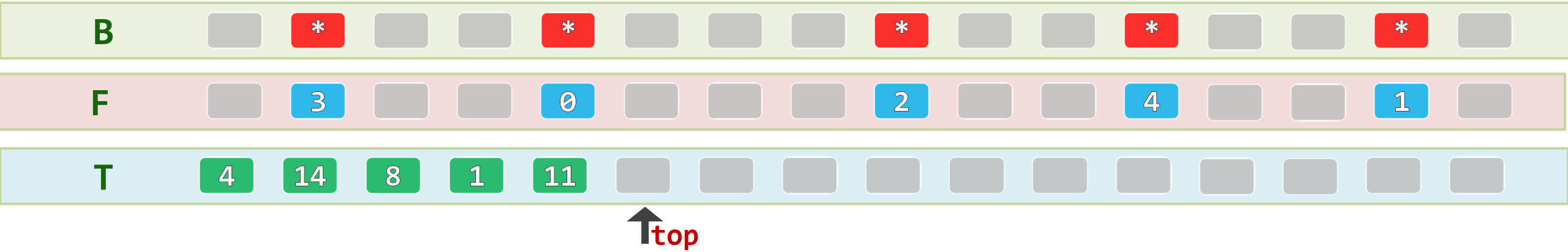


Bitmap举例:set(1);set(11);test(1);test(3)

❖ `void Bitmap::set( Rank k ) { if ( !test( k ) ) { T[top] = k; F[k] = top++; } }`



❖ `bool Bitmap::test( Rank k ) { return (F[k] < top) && (k == T[F[k]]); }`



Bitmap举例: clear(14)

❖ `void Bitmap::clear( Rank k )`

```
{ if ( test( k ) && ( --top ) ) { F[T[top]] = F[k]; T[F[k]] = T[top]; } }
```

